



香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

ELEC 3310

Digital Fundamentals and System Design

Wenchao Miao

Outline Lecture 4 Combinational Logic



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

1 Learning Outcomes

2 Encoders & Priority Encoders

3 Multiplexers (MUX)

4 Demultiplexers (DEMUX)

5 Parity & Error Detection

6 Timing & Glitch Analysis

7 Traffic Signal Controller Design

1. Learning Outcomes

- 1. Describe an encoder and a decimal-to-binary priority encoder**
- 2. Explain the basic operation of multiplexers and demultiplexers**
- 3. Expand a multiplexer to handle more data inputs**
- 4. Implement logic functions through multiplexers**
- 5. Describe parity method for error detection**
- 6. Determine if any error in a code based on a parity bit**
- 7. Describe odd parity and even parity concepts**
- 8. Implement circuits for generating and checking odd/even parity**
- 9. Design a traffic signal controller (combinational part)**

Main Content

1. Decimal-to-Binary Encoders and Applications

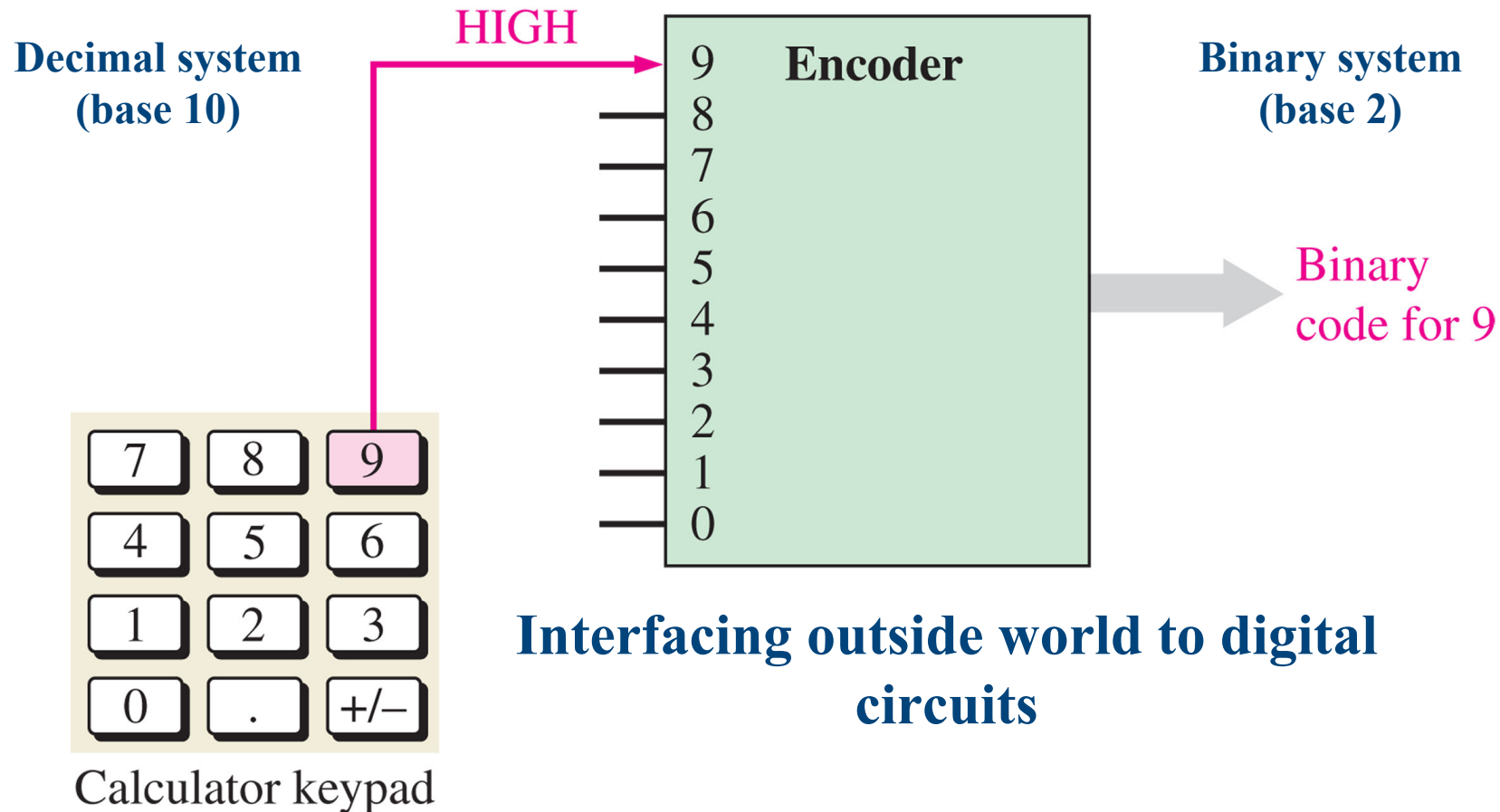
- Basic Encoder Operation: Decimal-to-Binary Conversion
- BCD Encoder Truth Table and Logic Diagram
- Priority Encoder (74HC147)
- Simplified Keyboard Encoder Design

2. Encoders & Priority Encoders

(1) Basic Encoder Operation: Decimal-to-Binary Conversion

9 in base 10 = 1001 in base 2

$$(9)_{10} = (1001)_2$$



2. Encoders & Priority Encoders

(2) BCD Encoder Truth Table and Logic Diagram

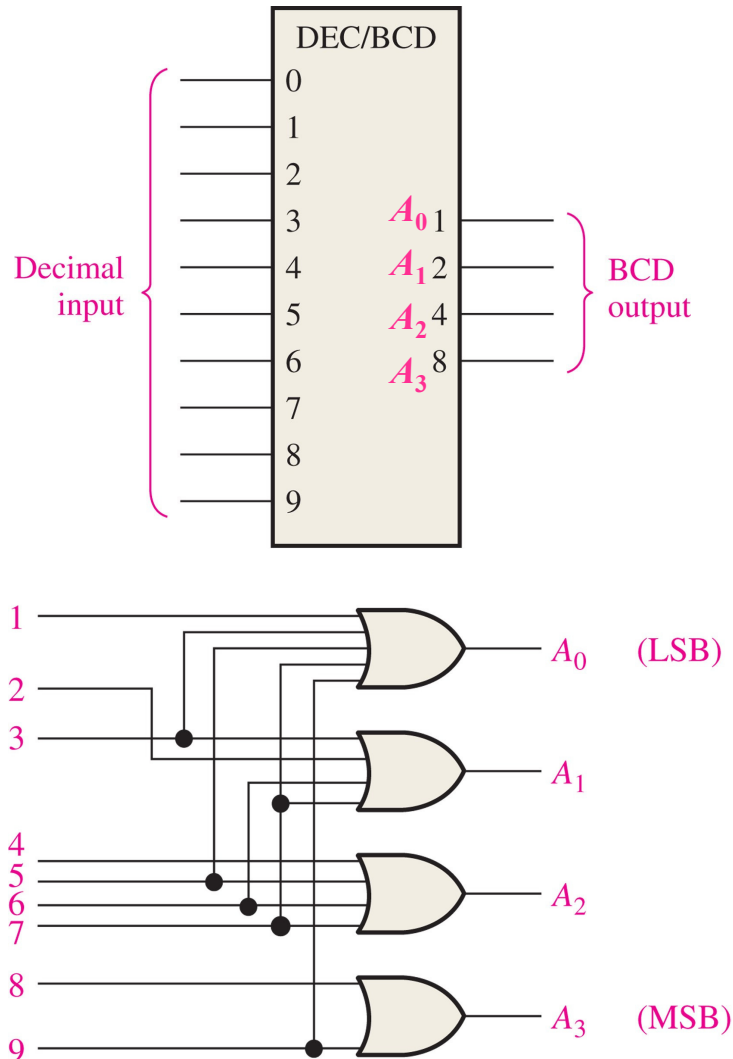


TABLE 6-6

Decimal Digit	BCD Code			
	A_3	A_2	A_1	A_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

A_0 = Inputs 1 OR 3 OR 5 OR 7 OR 9

A_1 = Inputs 2 OR 3 OR 6 OR 7

A_2 = Inputs 4 OR 5 OR 6 OR 7

A_3 = Inputs 8 OR 9

Input 0 is not needed because the BCD outputs are all LOW when there are no HIGH inputs.

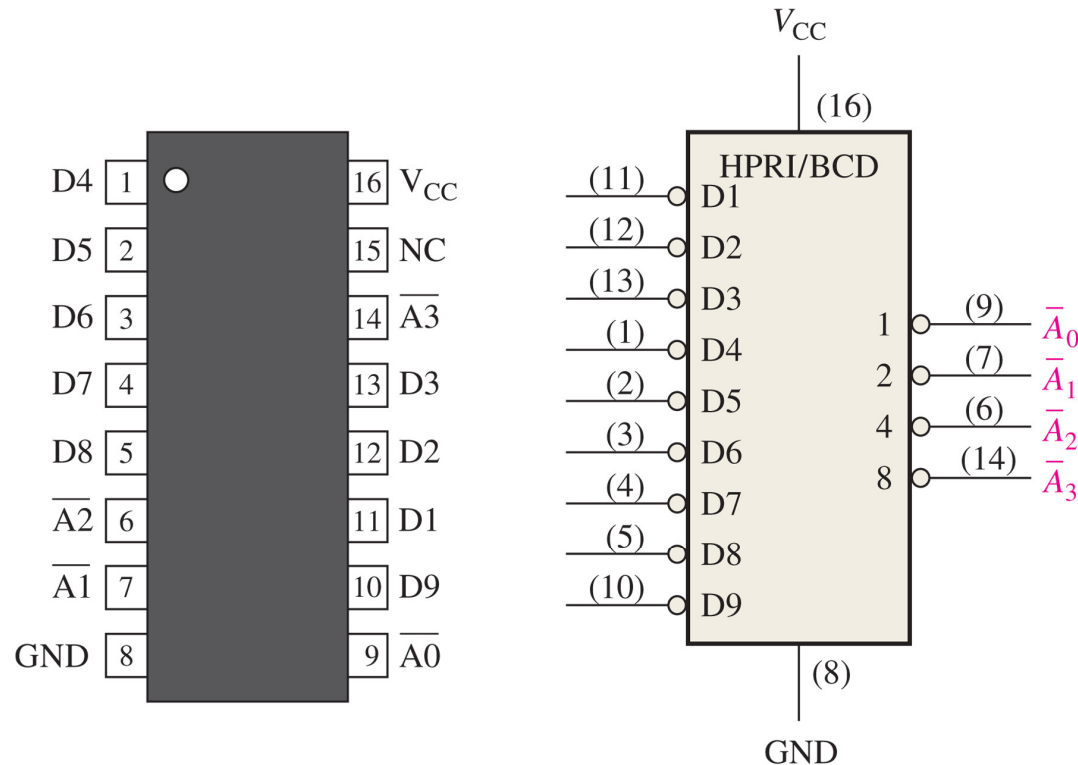
2. Encoders & Priority Encoders

(3) Priority Encoder (74HC147): Ignoring Lower-Order Inputs

The 74HC147 decimal-to-BCD encoder
HPRI means highest value input has priority

The encoder will ignore any other lower-order active inputs

Example: For both the 6 and the 3 inputs active, the BCD output is 0110 (which represents decimal 6)



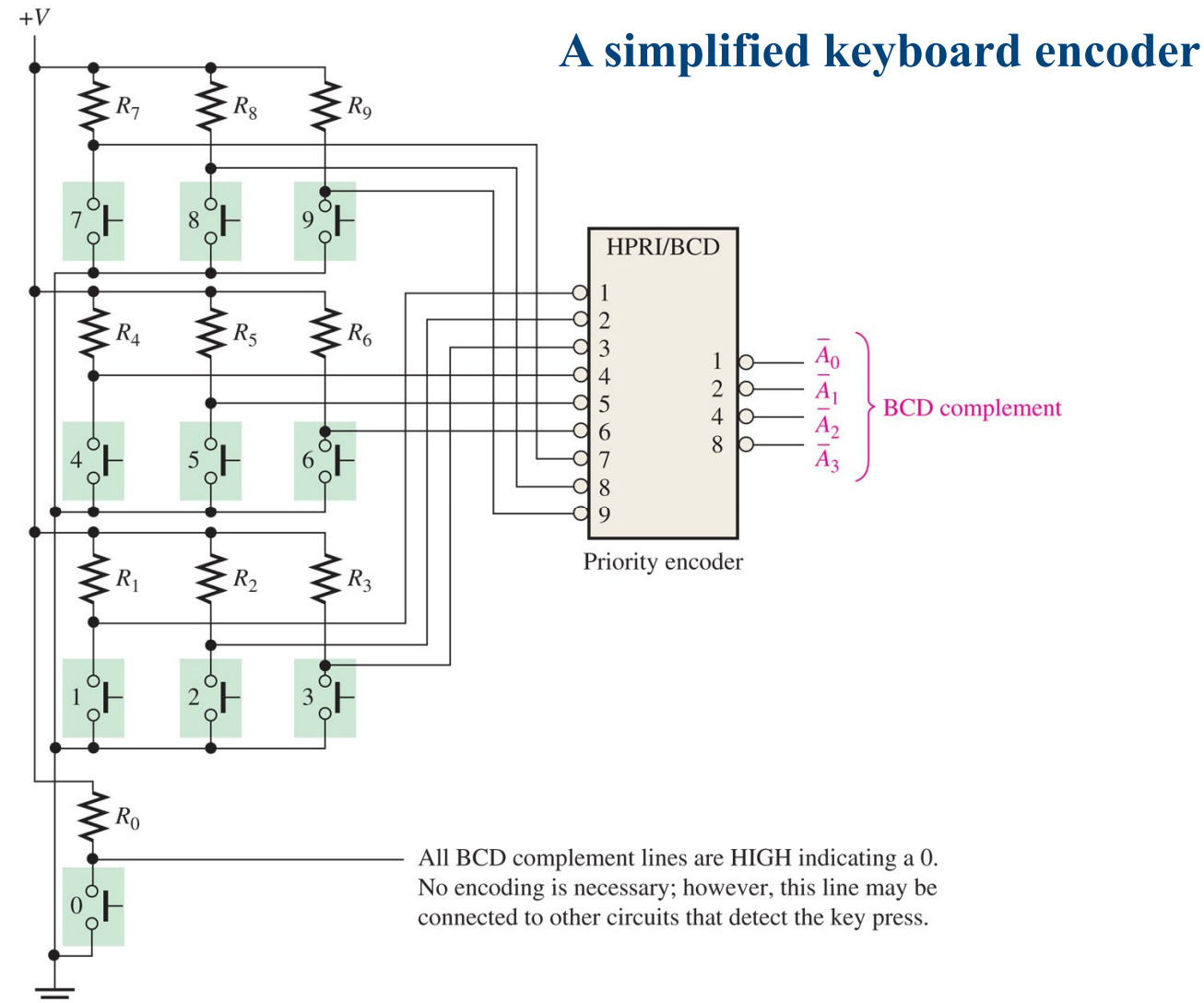
(a) Pin diagram

(b) Logic diagram

2. Encoders & Priority Encoders



(4) Simplified Keyboard Encoder Design



Main Content

1. MUX Fundamentals: Switching Inputs to Outputs
2. 4-to-1 MUX Design: Logic Statement, Symbol, and Timing Diagram
3. Logic Function Implementation with MUX
4. Advanced MUX Applications: Time-Division Multiplexing

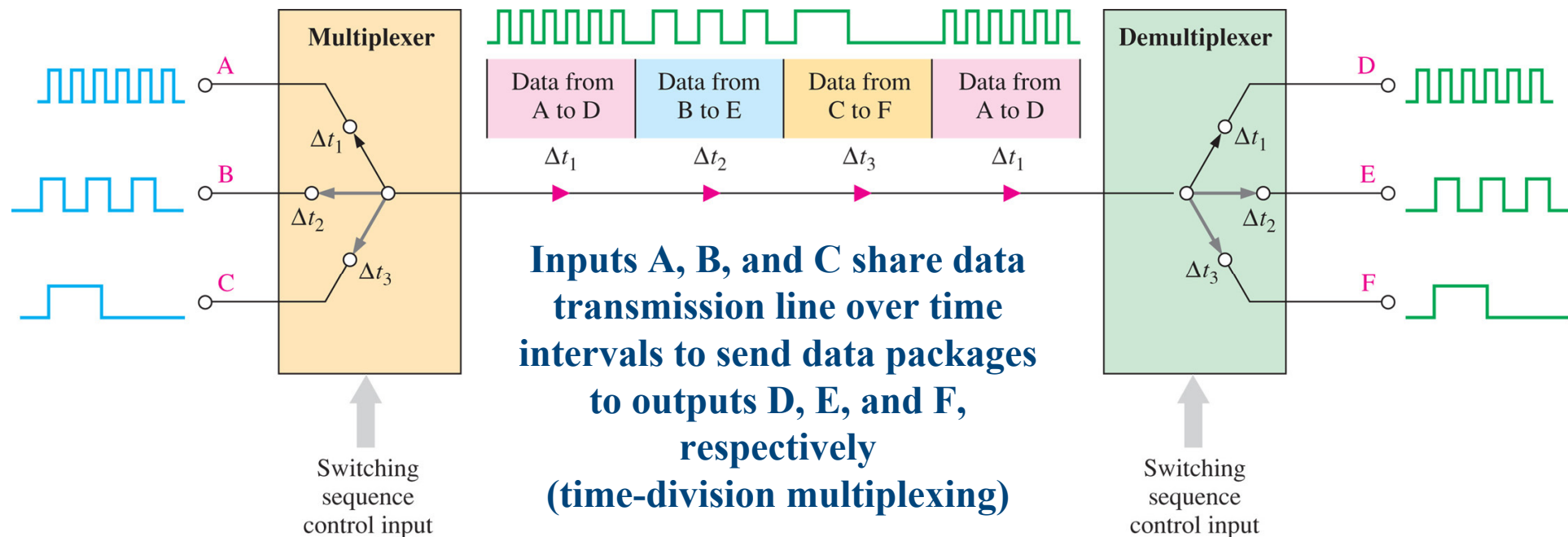
3. Multiplexers (MUX)

1. MUX Fundamentals: Switching Inputs to Outputs

Both are electronic switches

Multiplexer switches
many inputs to one output

Demultiplexer switches
one input to many outputs



3. Multiplexers (MUX)

2. 4-to-1 MUX Design: Logic Statement, Symbol, and Timing Diagram

Design a 4-to-1 MUX !

It requires 6-input Truth table (4 data and **2 data select**); $2^6 = 64$ rows!

So we skip truth table and write a logic statement as in 4-bit comparator

$Y = D_0$ when $S_1 = 0$ AND $S_0 = 0$, OR

$Y = D_1$ when $S_1 = 0$ AND $S_0 = 1$, OR

$Y = D_2$ when $S_1 = 1$ AND $S_0 = 0$, OR

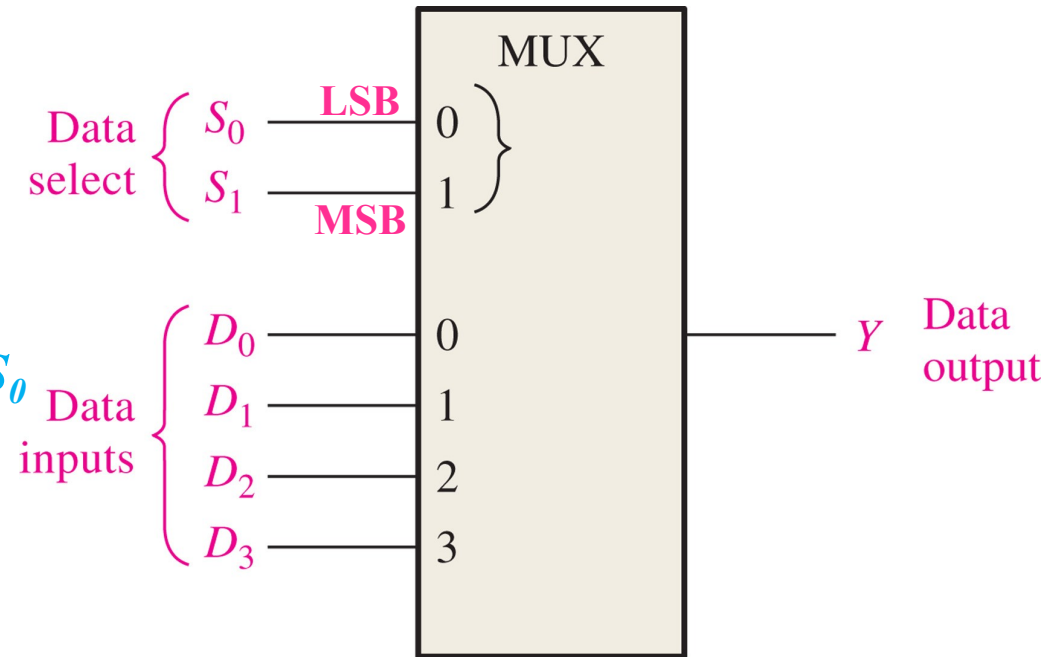
$Y = D_3$ when $S_1 = 1$ AND $S_0 = 1$

If you write Minterms expansion:

$$Y = D_0 \bar{S}_1 \bar{S}_0 + D_1 \bar{S}_1 S_0 + D_2 S_1 \bar{S}_0 + D_3 S_1 S_0$$

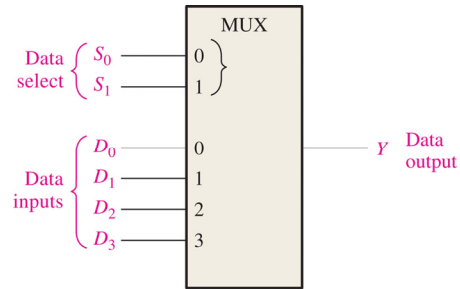
Logic symbol for a 1-of-4 data selector/multiplexer (MUX).

4-to-1 MUX



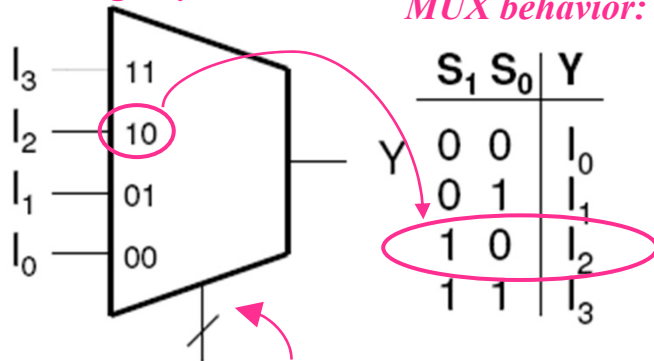
3. Multiplexers (MUX)

2. 4-to-1 MUX Design: Logic Statement, Symbol, and Timing Diagram



Data inputs are represented by *D* or *I*

Another common
MUX logic symbol:



MUX behavior:

S ₁	S ₀	Y
0	0	I ₀
0	1	I ₁
1	0	I ₂
1	1	I ₃

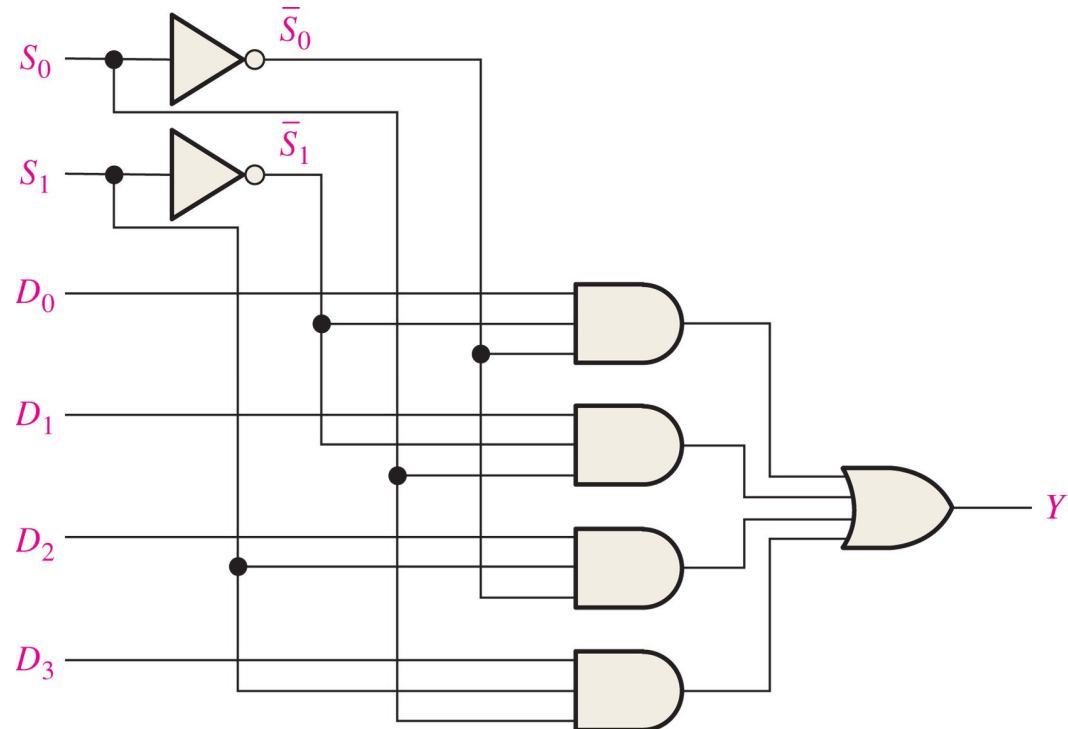
S(1:0)

Slash indicates multi-bit inputs
(2 inputs in this case: S₀; S₁)

if S = 00, then Y = I₀
if S = 01, then Y = I₁
if S = 10, then Y = I₂
if S = 11, then Y = I₃

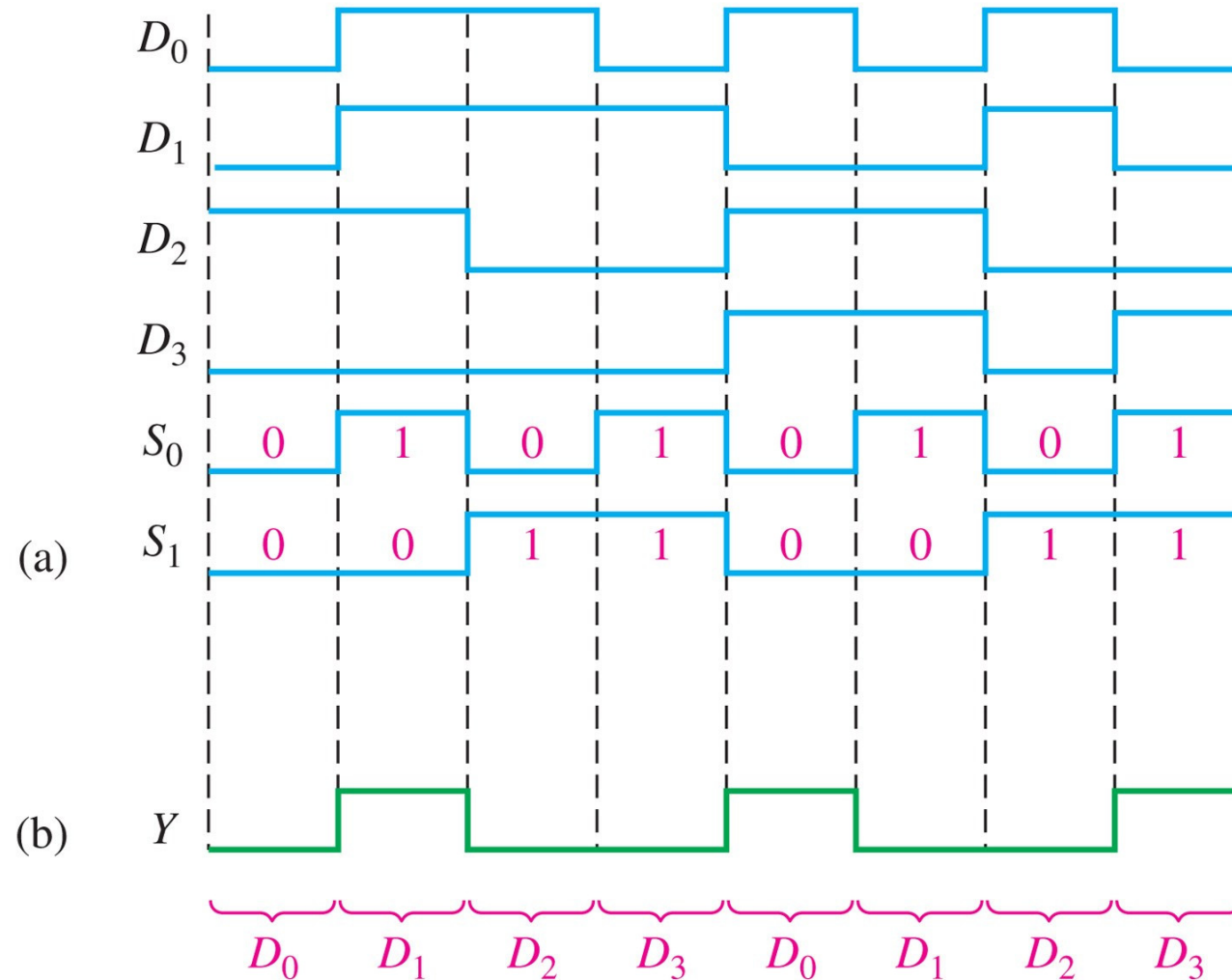
$$Y = D_0 \bar{S}_1 \bar{S}_0 + D_1 \bar{S}_1 S_0 + D_2 S_1 \bar{S}_0 + D_3 S_1 S_0$$

Output is in the form of SOP which means that
you can use MUX to implement logic functions.



3. Multiplexers (MUX)

2. 4-to-1 MUX Design: Logic Statement, Symbol, and Timing Diagram



3. Logic Function Implementation with MUX

(1) 3-Input and 4-Input Function Examples

Implementing a 3-input function using an 8-to-1 MUX

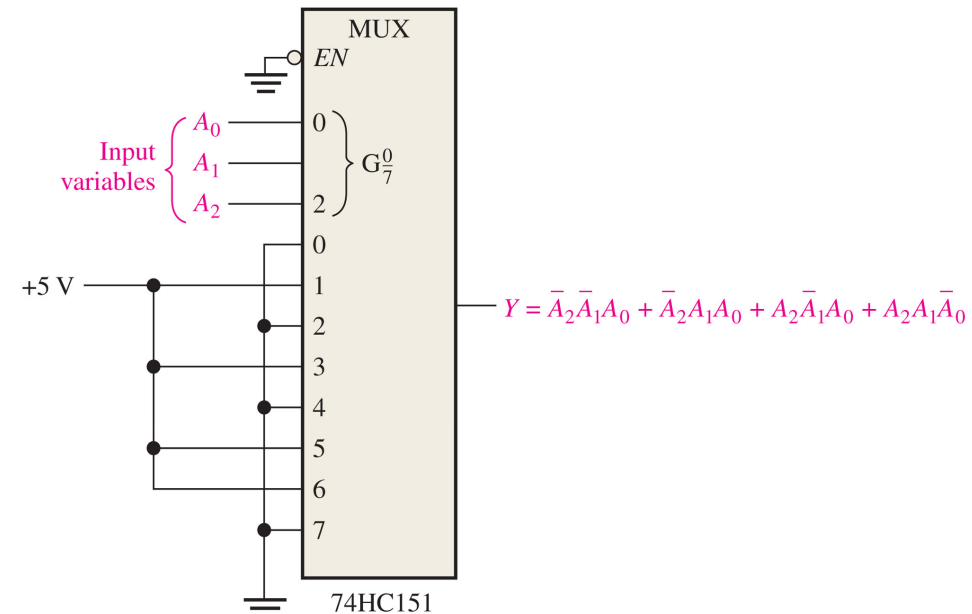
If the MUX is the right size (the select inputs match the function inputs)

Step 1: Feed the function inputs to the MUX select inputs (watch out for MSB)

Step 2: Feed the function output to the MUX data inputs

TABLE 6-9

Inputs			Output
A_2	A_1	A_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0



3. Multiplexers (MUX)



Implementing 4-input function using 8-to-1 MUX

If the MUX is smaller (less select inputs than the inputs of the function)

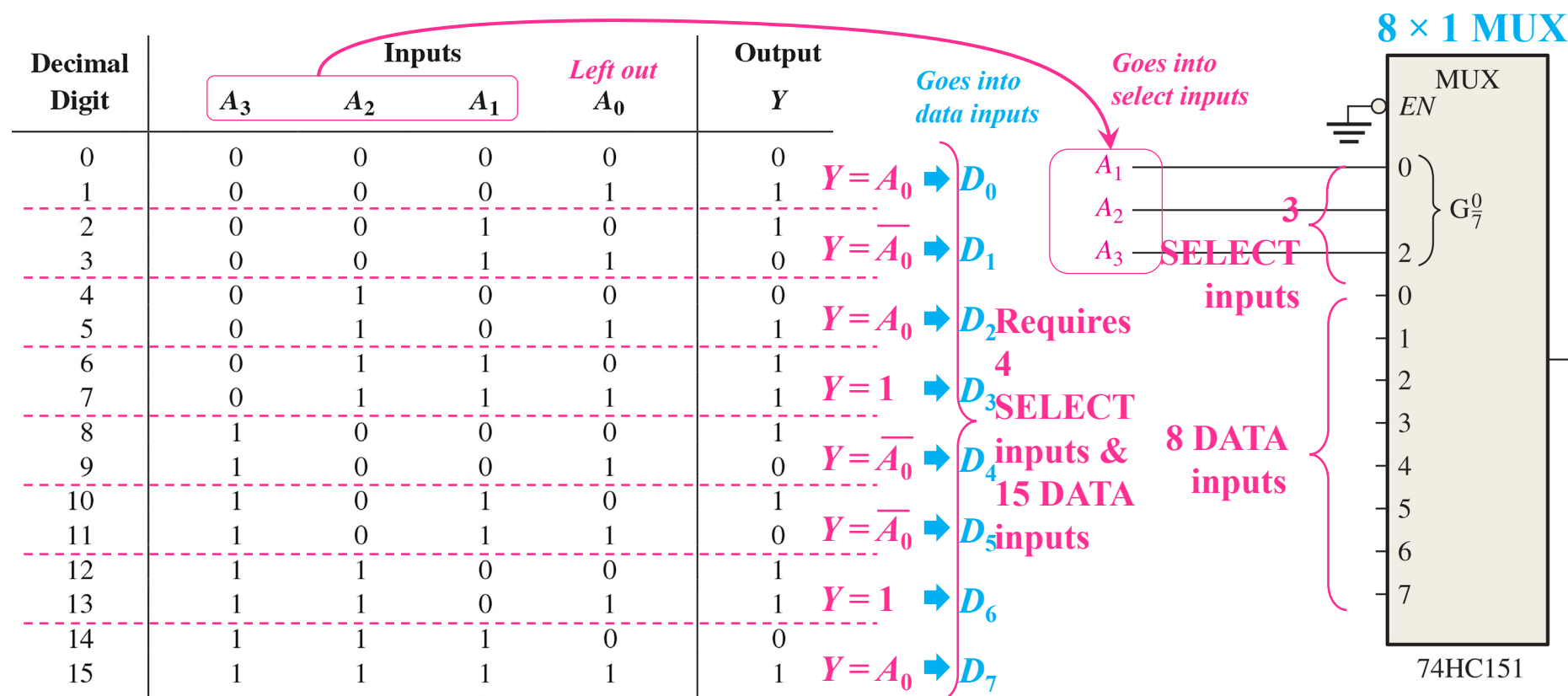
Step 1: Feed the function inputs to the MUX select inputs **starting from MSB**

some function inputs will be left out (will not fit)

Step 2: Express the function output in terms of the function input(s) left out after Step 1

partition the truth table according to the inputs fed to the MUX select inputs

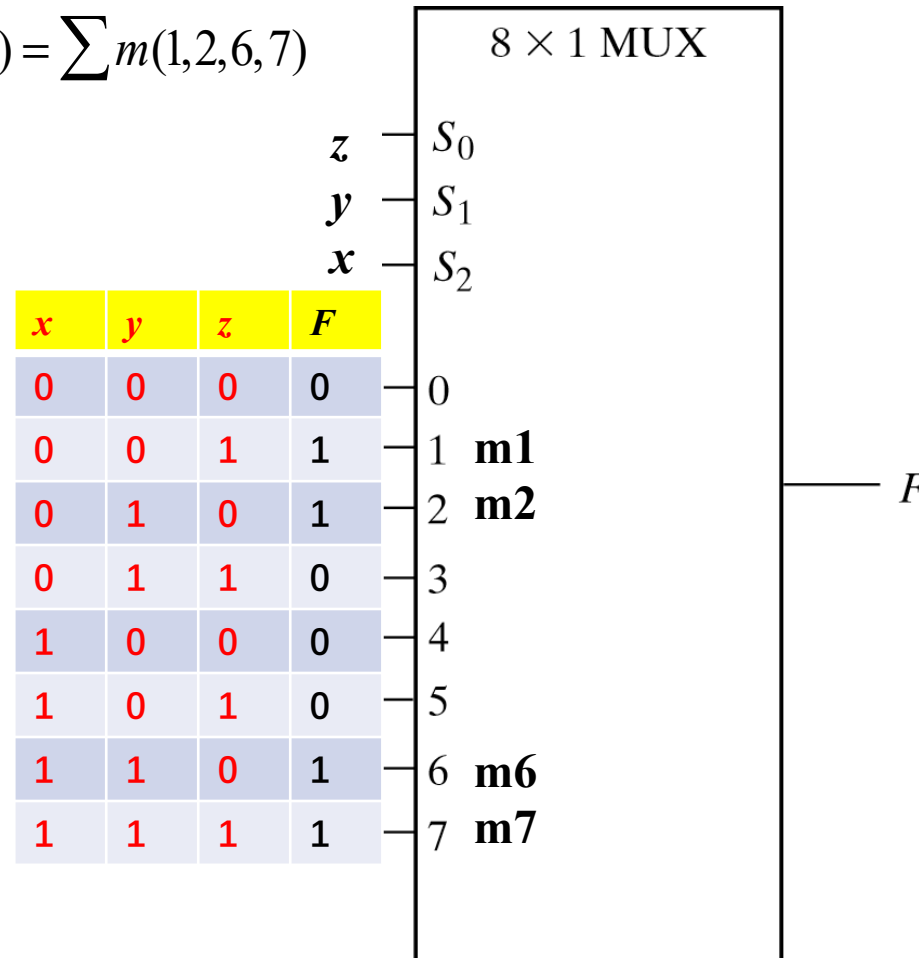
Step 3: Feed the expression within each partition to the corresponding data input



3. Multiplexers (MUX)

(1) 3-Input and 4-Input Function Examples

$$F(x,y,z) = \sum m(1,2,6,7)$$

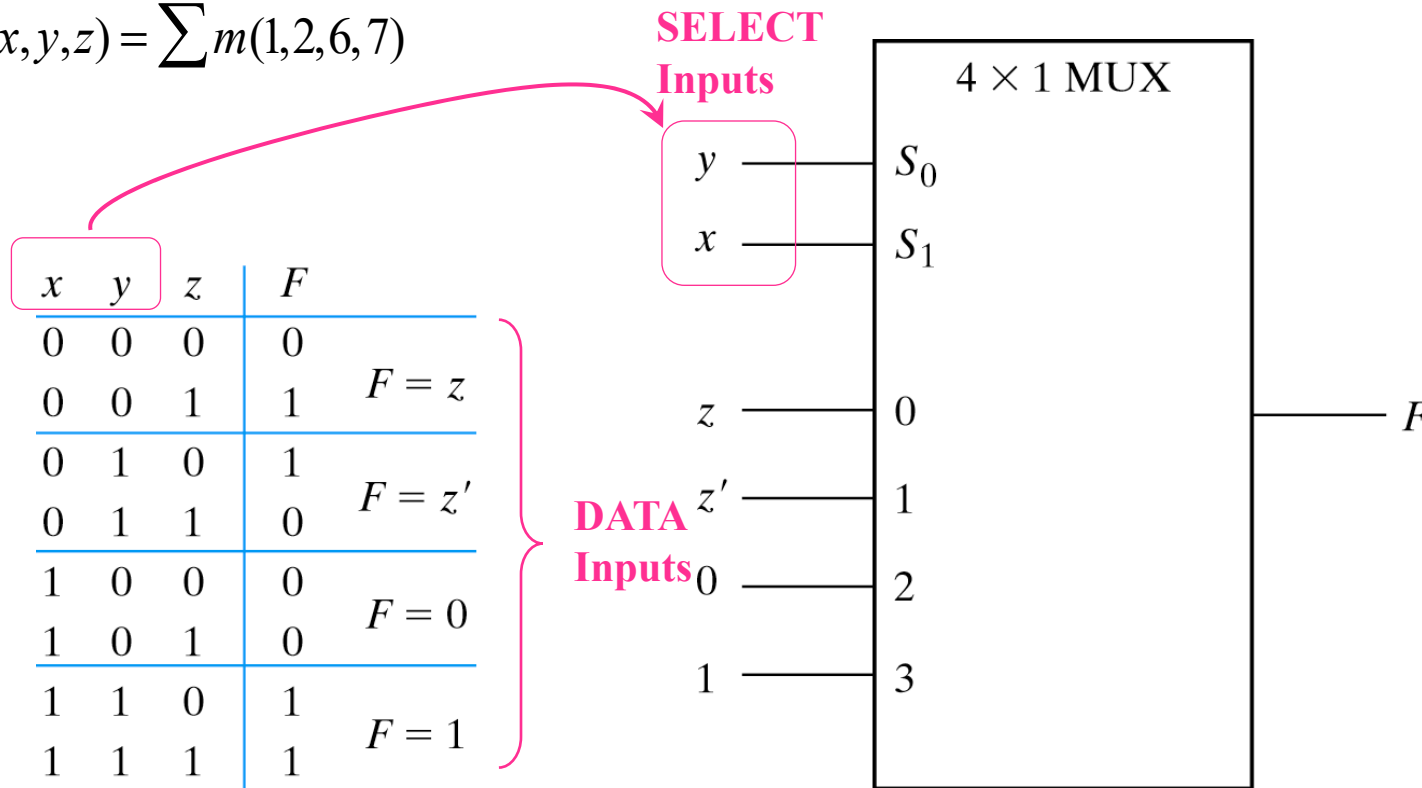


3. Multiplexers (MUX)



(1) 3-Input and 4-Input Function Examples

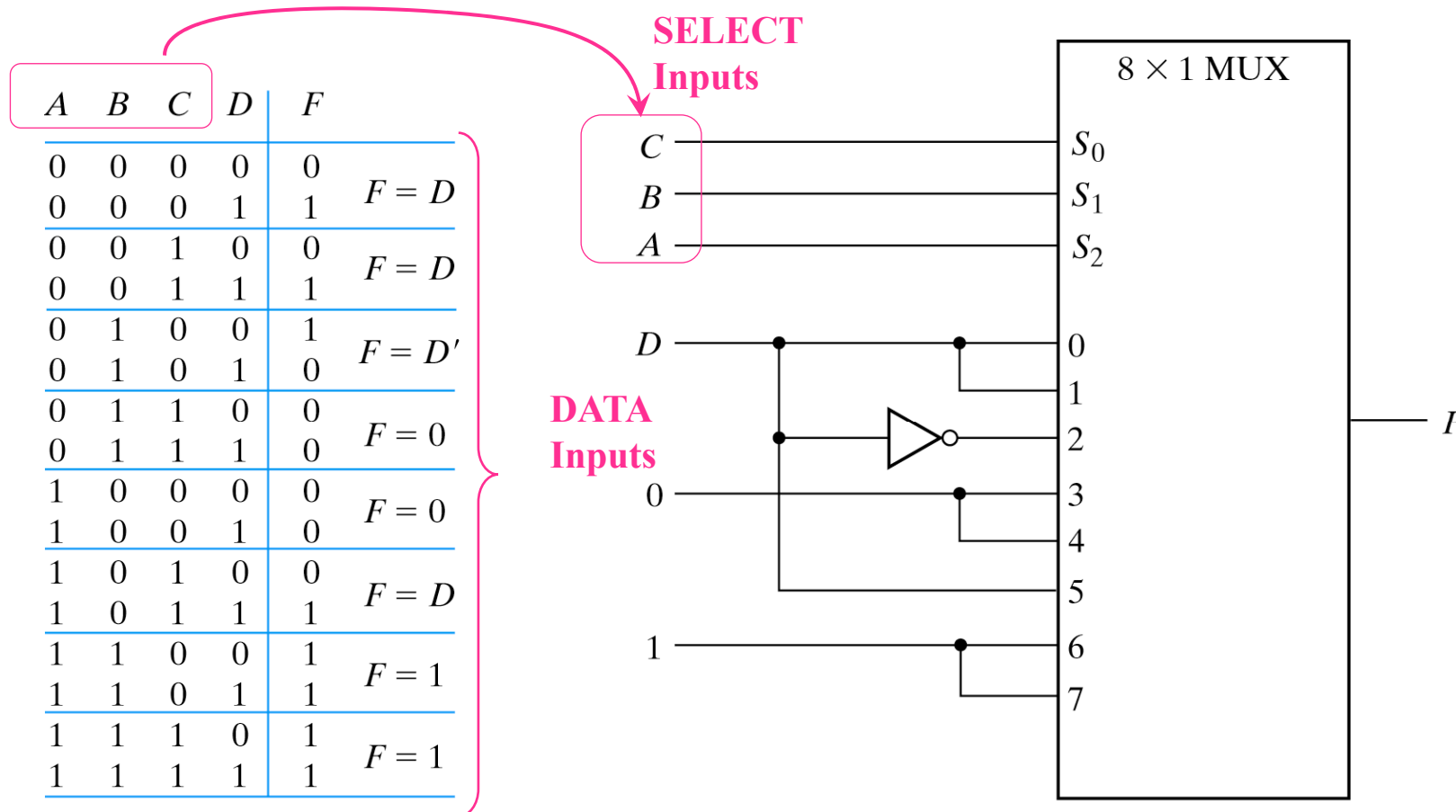
$$F(x,y,z) = \sum m(1,2,6,7)$$



3. Multiplexers (MUX)

(1) 3-Input and 4-Input Function Examples

$$F(A,B,C,D) = \sum m(1,3,4,11,12,13,14,15)$$



(2) 8×1 and 4×1 MUX Configurations

A	B	C	D	F	
0	0	0	0	1	F = 1
0	0	0	1	1	
0	0	1	0	0	F = D
0	0	1	1	1	
0	1	0	0	0	F = 0
0	1	0	1	0	
0	1	1	0	1	F = 1
0	1	1	1	1	
1	0	0	0	1	F = D'
1	0	0	1	0	
1	0	1	0	0	F = D
1	0	1	1	1	
1	1	0	0	1	F = D'
1	1	0	1	0	
1	1	1	0	1	F = D'
1	1	1	1	0	

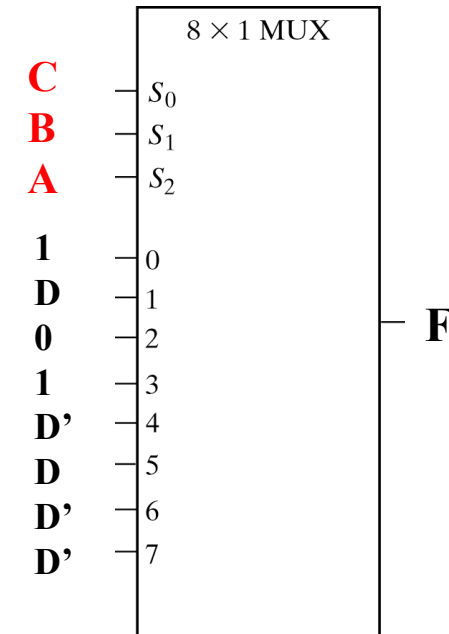
$$F(A,B,C,D) = \sum m(0,1,3,6,7,8,11,12,14)$$

A, B, C assigned to SELECT inputs!

**Partition the truth table accordingly: in each partition
→ A, B, C should remain same**

**Write a function for each partition using SOP or POS
of remaining literals (e.g. D)**

**DATA
Inputs**



4-Input Logic Function by 8×1 MUX

(2) 8×1 and 4×1 MUX Configurations

$$F(A,B,C,D) = \sum m(0,1,3,6,7,8,11,12,14)$$

A	B	C	D	F
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1

$$F = C' + D$$

0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1

$$F = C$$

1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1

$$F = C \text{ XNOR } D$$

1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

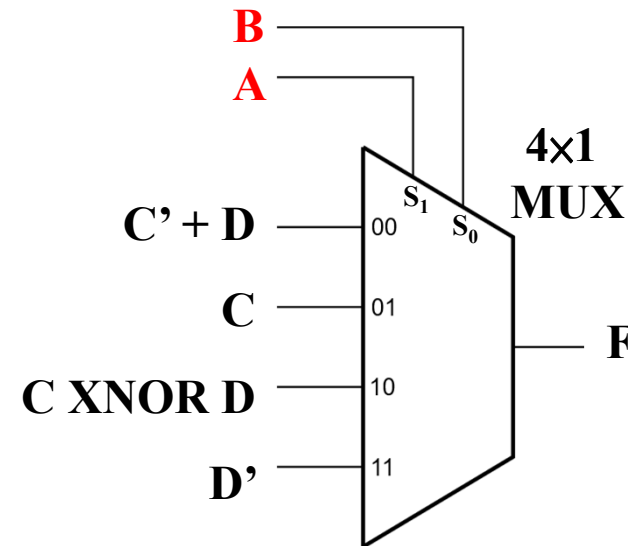
$$F = D'$$

A, B, assigned to SELECT inputs!

Partition the truth table accordingly: in each partition $\rightarrow$ A, B should remain same

Write a function for each partition using SOP or POS of remaining literals (e.g. C, D)

**DATA
Inputs**



4-Input Logic Function by 4×1 MUX

(3) Boolean Simplifications for 2×1 MUX

$$F(A,B,C,D) = \Sigma m(0,1,3,6,7,8,11,12,14)$$

A	B	C	D	F
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

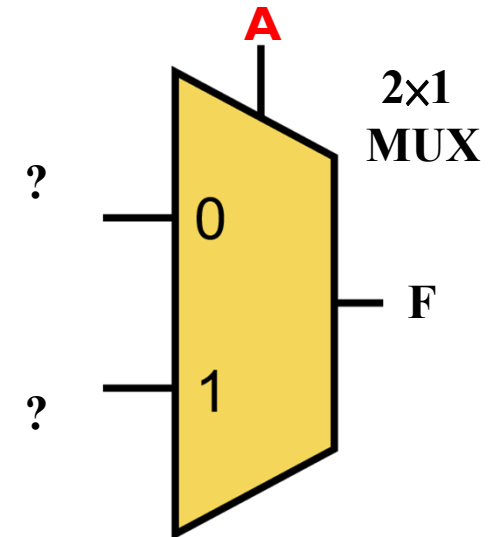
F = ?

F = ?

Input A assigned to SELECT input!

Partition the truth table accordingly: in each partition → Input A should remain same

Write a function for each partition using SOP or POS of remaining literals (e.g. B, C, D)



(3) Boolean Simplifications for 2×1 MUX

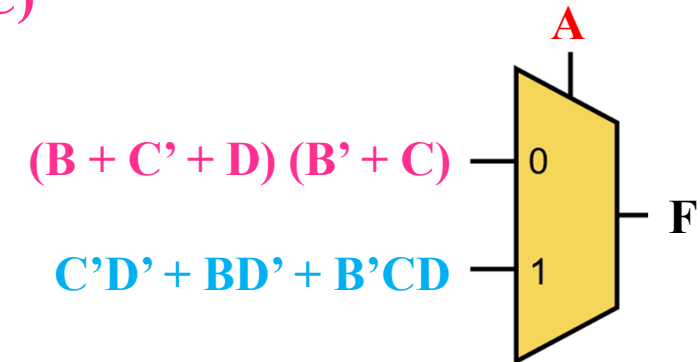
A	B	C	D	F
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

$$F(A,B,C,D) = \sum m(0,1,3,6,7,8,11,12,14)$$

$$F = (B + C' + D) (B' + C + D) (B' + C + D')$$

Applying Boolean Algebra simplifies the expression:

$$F = (B + C' + D) (B' + C)$$



$$F = B'C'D' + B'CD + BC'D' + BCD'$$

Applying Boolean Algebra simplifies to:

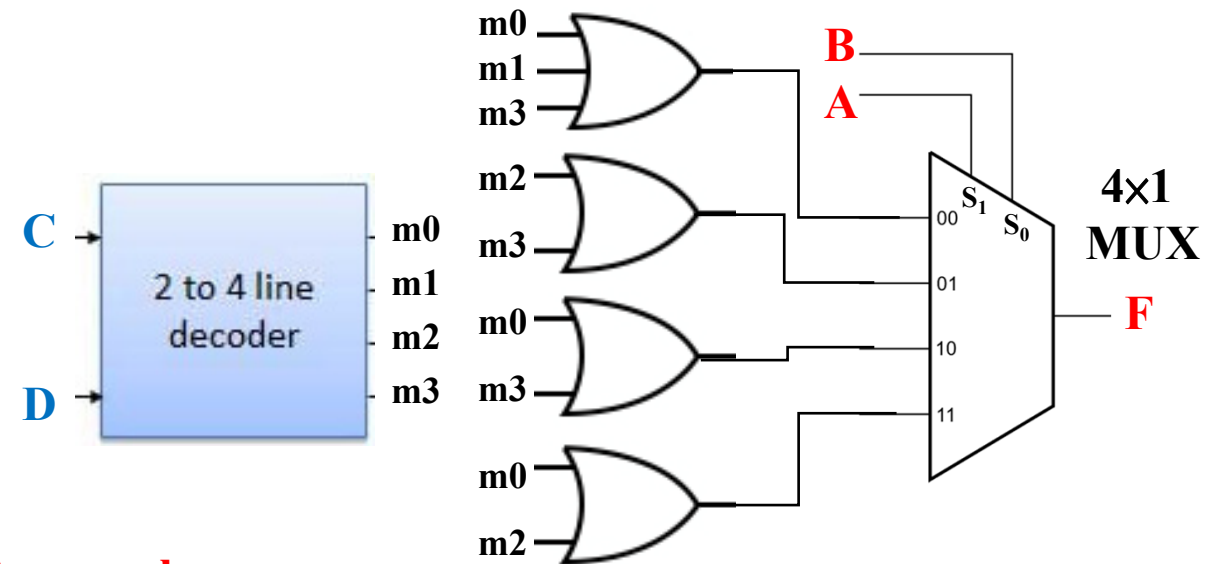
$$F = C'D' + BD' + B'CD$$

Logic F by 4 × 1 MUX & 2:4 DECODER

A	B	C	D	F	
0	0	0	0	1	m0
0	0	0	1	1	m1
0	0	1	0	0	m2
0	0	1	1	1	m3
0	1	0	0	0	m0
0	1	0	1	0	m1
0	1	1	0	1	m2
0	1	1	1	1	m3
1	0	0	0	1	m0
1	0	0	1	0	m1
1	0	1	0	0	m2
1	0	1	1	1	m3
1	1	0	0	1	m0
1	1	0	1	0	m1
1	1	1	0	1	m2
1	1	1	1	0	m3

$$F(A,B,C,D) = \sum m(0,1,3,6,7,8,11,12,14)$$

Decoder is a minterm generator and also used to implement logic functions, however needs an OR gate to sum minterms



Approach:

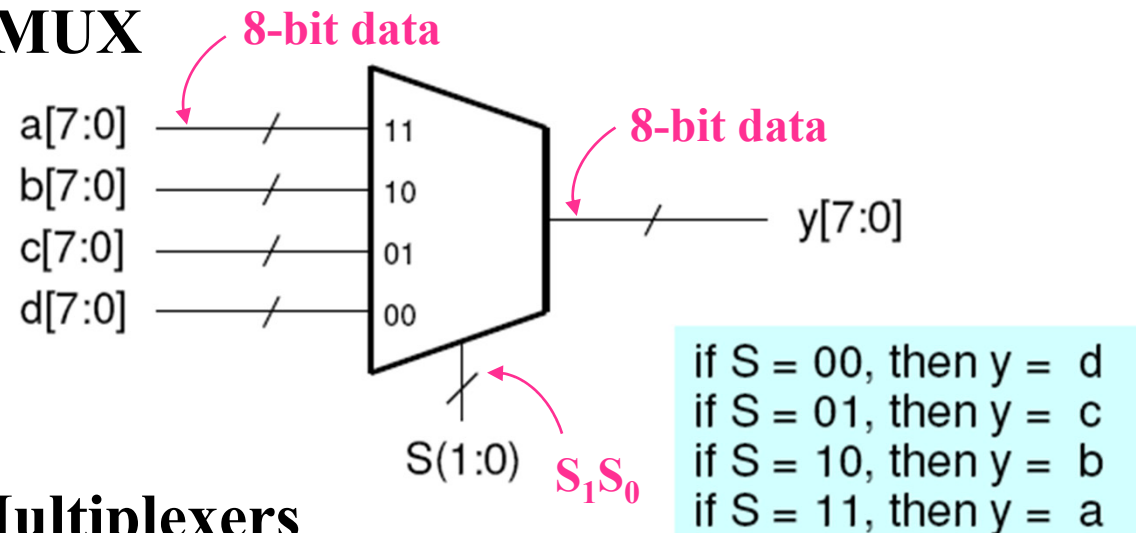
- Use 4x1 MUX to partition the Truth Table as done before
- Use 2:4 Decoder to generate minterms for each partition
- Use 4 OR gates to produce SOP for each partition
- Connect each OR gate output to MUX respective data input

(4) Multi-Bit MUX Applications

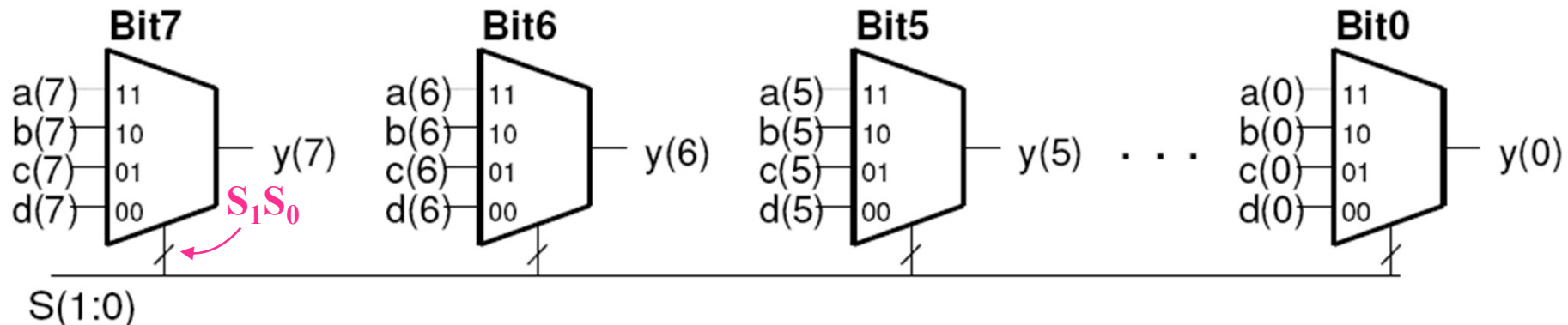
Multiplexers are often used to select groups of bits arranged in buses

Example: You can multiplex 8-bit data (1 byte) bus using either

- **4-to-1 8-bit MUX**



- **eight 1-bit Multiplexers**



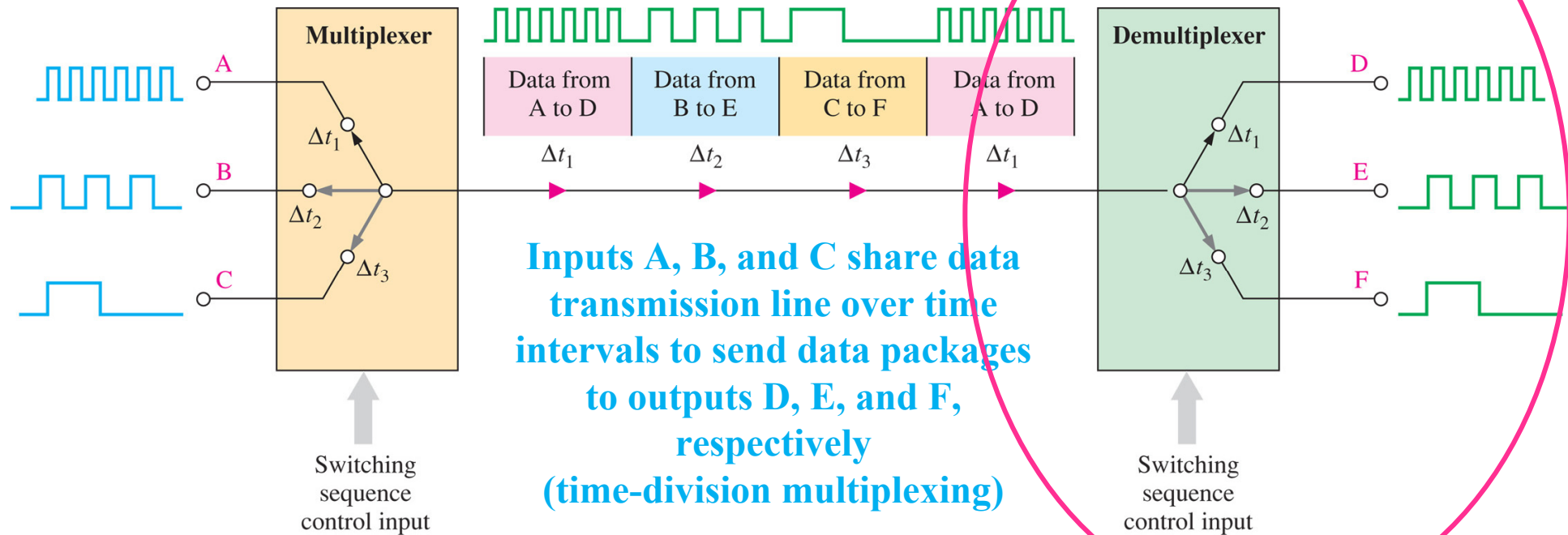
4. Demultiplexers (DEMUX)

Both are electronic switches

Demultiplexer reverses the
multiplexing function

Multiplexer switches
many inputs to one output

Demultiplexer switches
one input to many outputs



Main Content

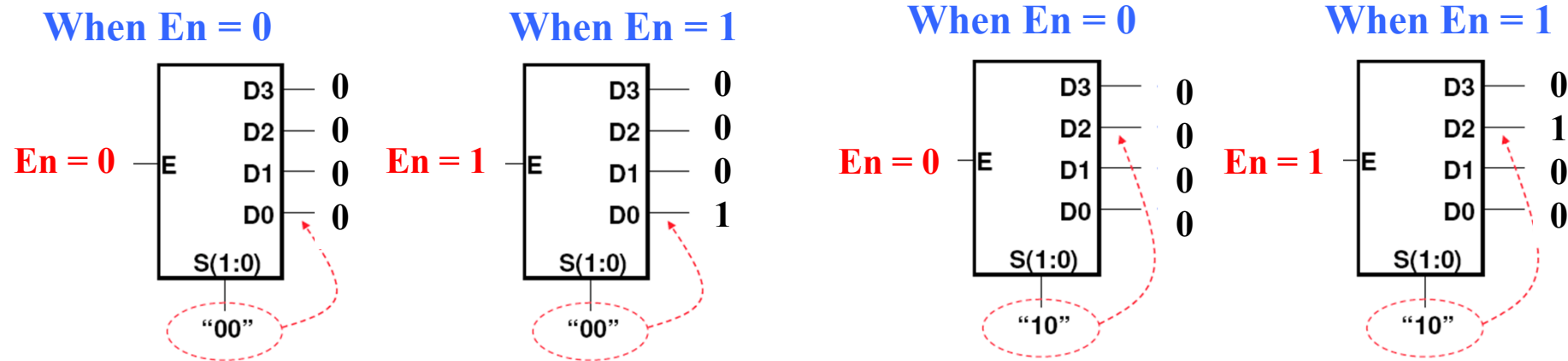
Demultiplexers and Their Role in Digital Systems

1. DEMUX as Decoder with Enable Input
2. 1-to-4 Line DEMUX Logic Diagram
3. Practical DEMUX Example and Timing
4. Transition to Parity & Error Detection

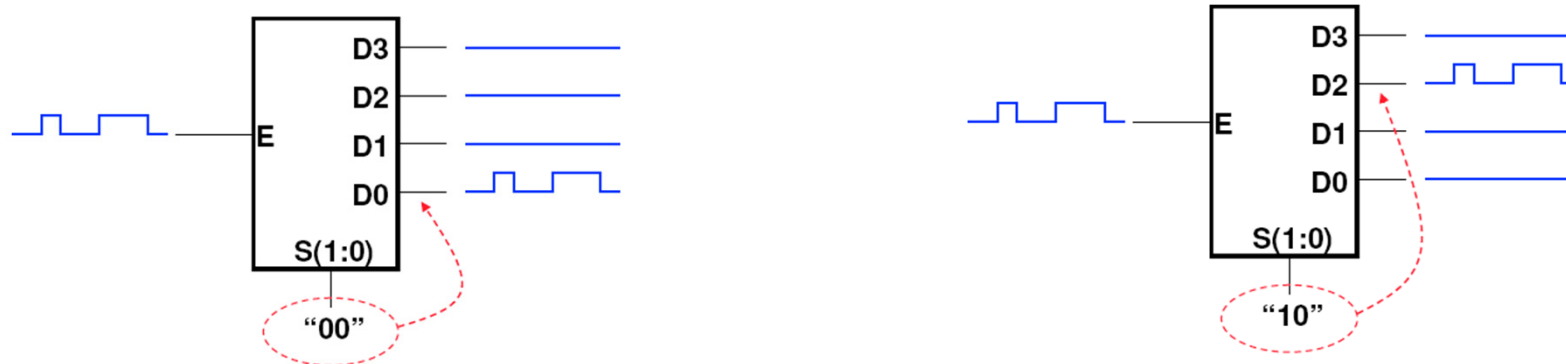
4.1 DEMUX as Decoder with Enable Input



- Decoder with Enable input works as a demultiplexer
 - When $En = 0$, all the outputs are zero.
 - When $En = 1$, only the selected output is 1 all the remaining outputs 0.



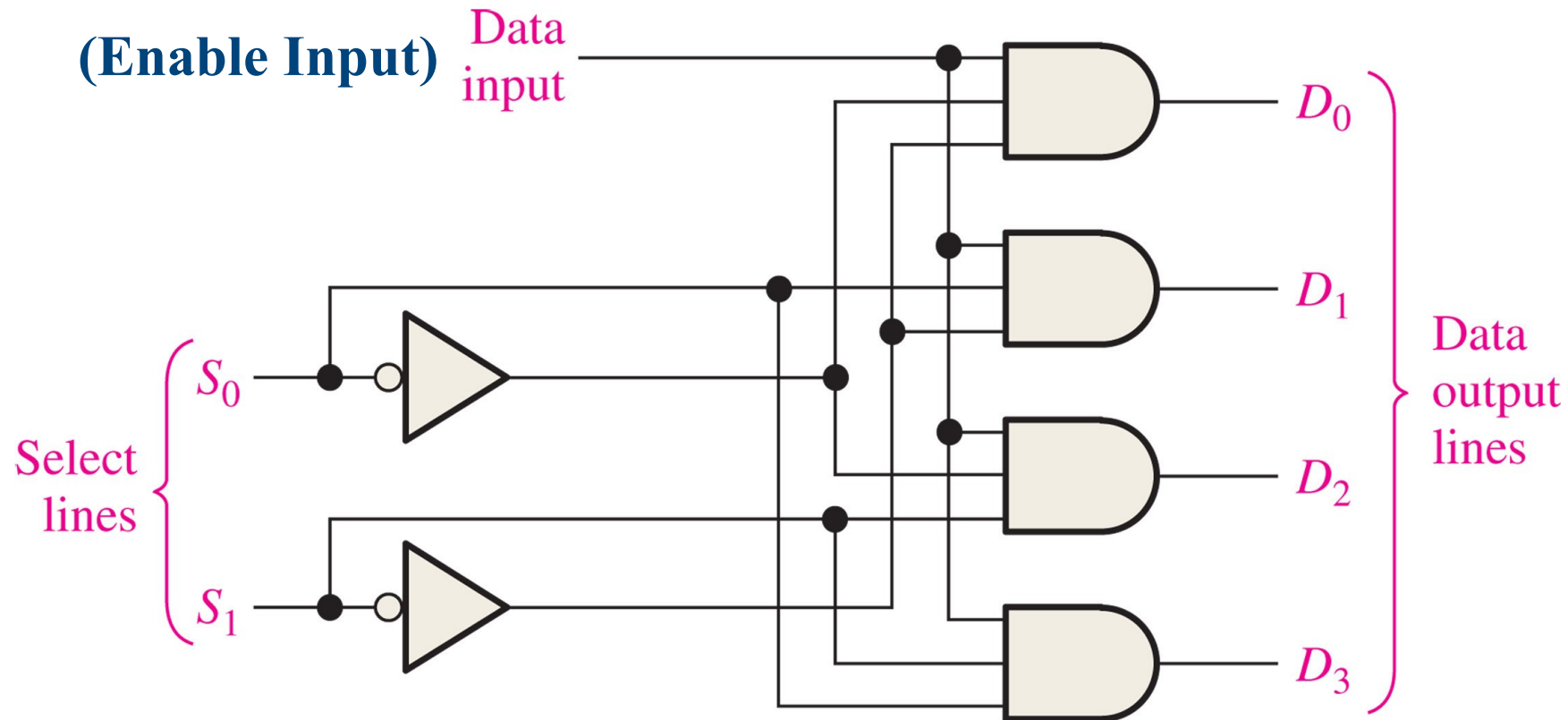
- DEMUX directs the pulses (signal) applied to ENABLE input into one of the output lines ($D_N...D_0$) identified by the select inputs (S)



4. Demultiplexers (DEMUX)

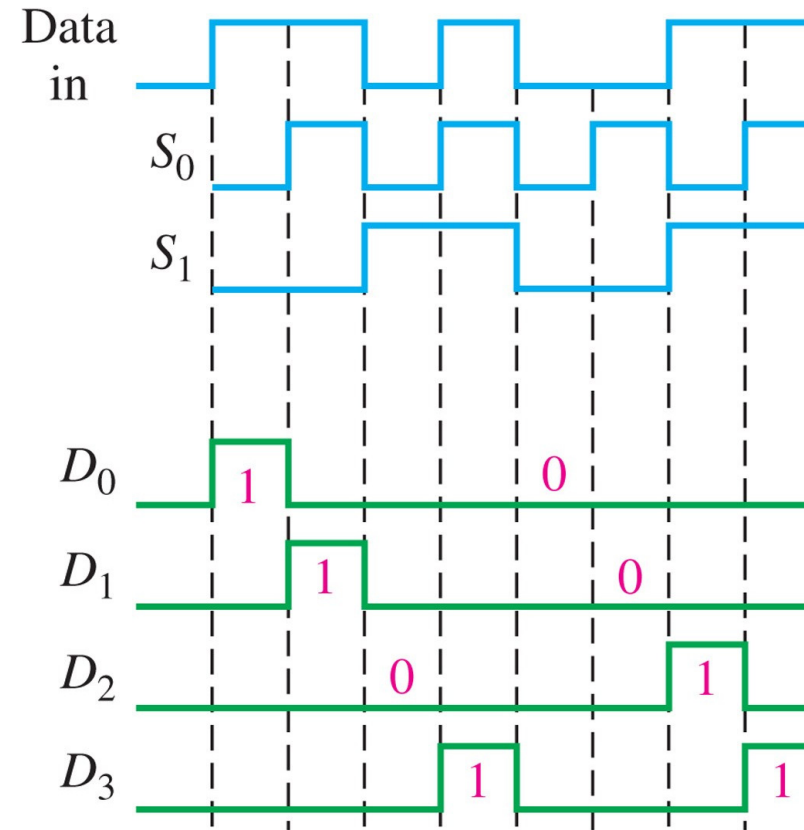
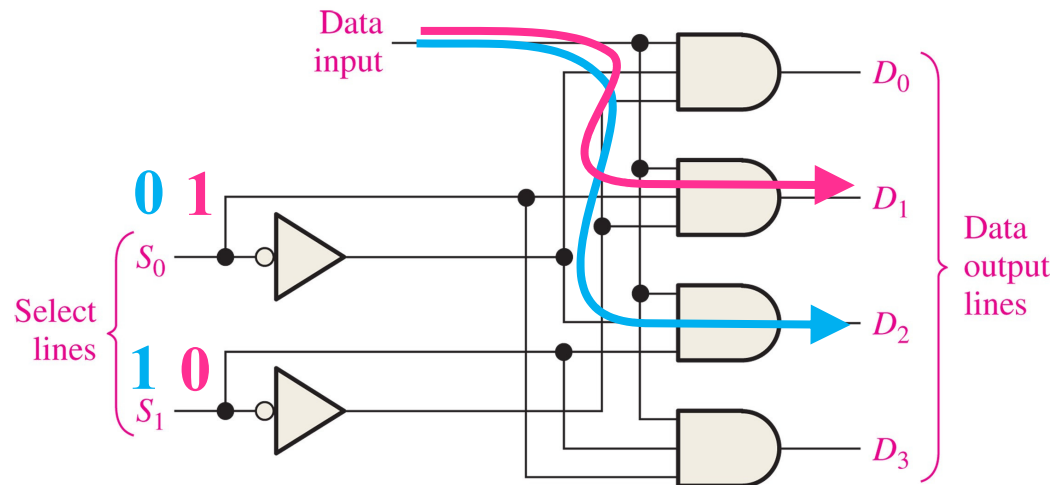
2. 1-to-4 Line DEMUX Logic Diagram

Also 2-to-4 Line Decoder with Enable Input



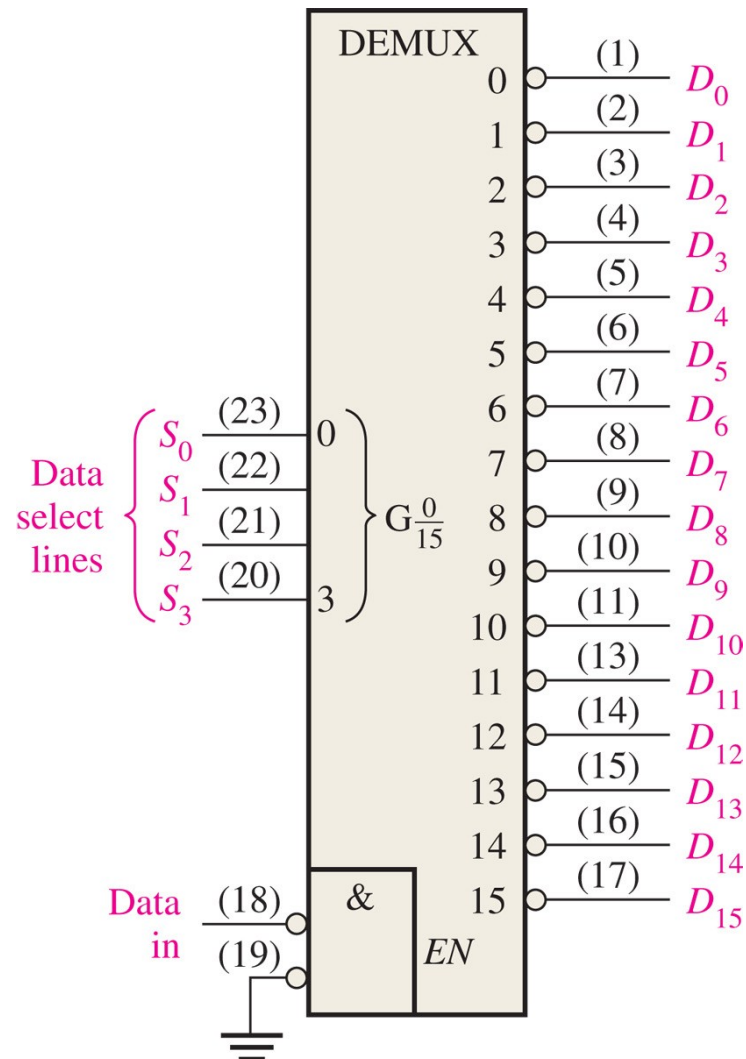
4. Demultiplexers (DEMUX)

2. 1-to-4 Line DEMUX Logic Diagram



4. Demultiplexers (DEMUX)

Decoder used as a Demultiplexer



Main Content

Ensuring Data Integrity with Parity

1. Parity Basics: Even/Odd Parity, XOR-Based Detection
2. IC Applications: 74HC280 Parity Generator/Checker and Serial Transmission

(1) Parity Basics: Even/Odd Parity, XOR-Based Detection

Error (undesired change in the bits) can occur in the transfer of digital codes due to electrical noise or circuit malfunction

0 becomes 1

1 becomes 0

Solution: Attach a parity bit to the code (group of information bits) to make the total number of 1s either even or odd depending on the system

An even (odd) parity bit makes the total number of 1s even (odd).

Parity error: change in a single bit or odd numbers of bits (very unlikely)

TABLE 2-8

The BCD code with parity bits.

Even Parity		Odd Parity	
<i>P</i>	BCD	<i>P</i>	BCD
0	0000	1	0000
1	0001	0	0001
1	0010	0	0010
0	0011	1	0011
1	0100	0	0100
0	0101	1	0101
0	0110	1	0110
1	0111	0	0111
1	1000	0	1000
0	1001	1	1001

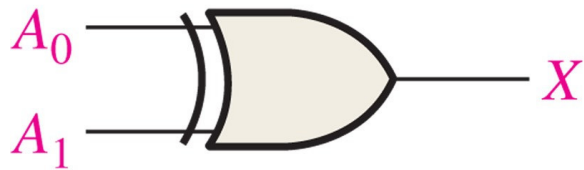
An even (odd) parity bit makes the total number of 1s even (odd)

The sum of an even number of 1s is always 0, and the sum of an odd number of 1s is always 1 (disregarding carries).

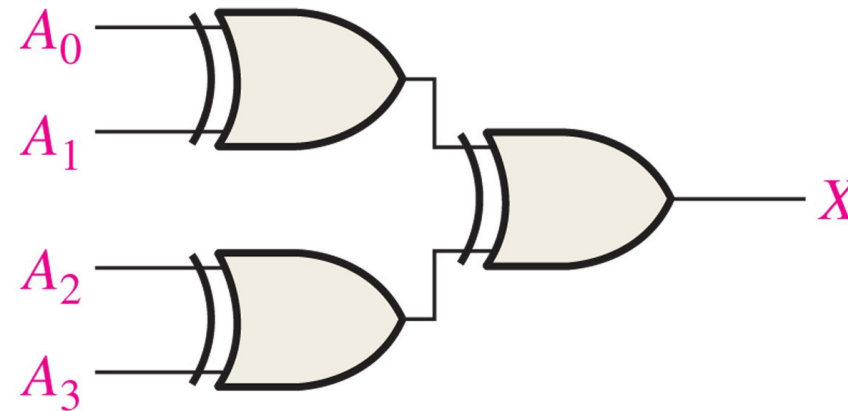
Hence XORs can be used in the generation/detection of parity.

Example: Logic for detecting a single-bit error in a 2-bit or a 4-bit code featuring even parity (X should be 0 if the received code is correct):

$X = 1$ means parity error



(a) Summing of two bits



(b) Summing of four bits

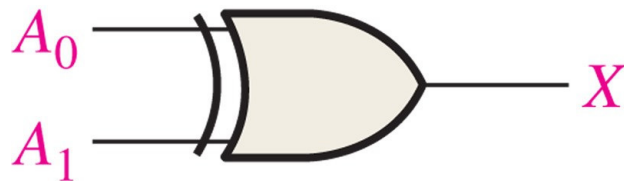
(1) Parity Basics: Even/Odd Parity, XOR-Based Detection

An even (odd) parity bit makes the total number of 1s even (odd).

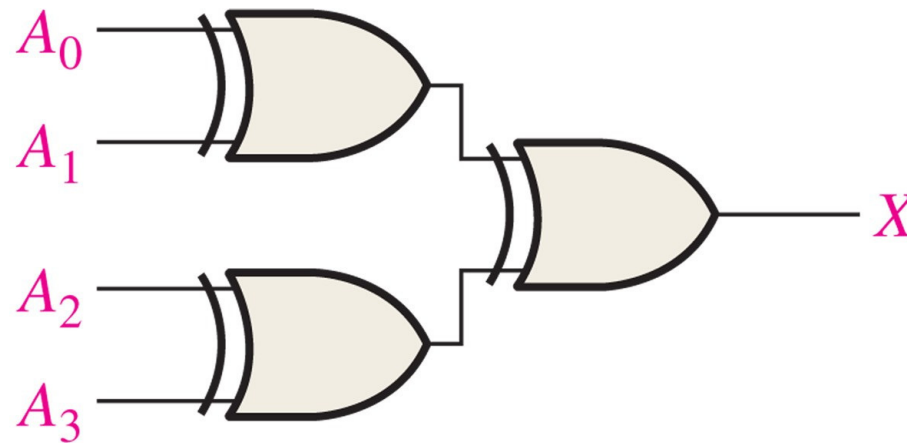
Example: Logic for generating a parity bit for a 2-bit or a 4-bit code to make an even parity (X is the parity bit):

$$XA_1A_0$$

$$XA_3A_2A_1A_0$$



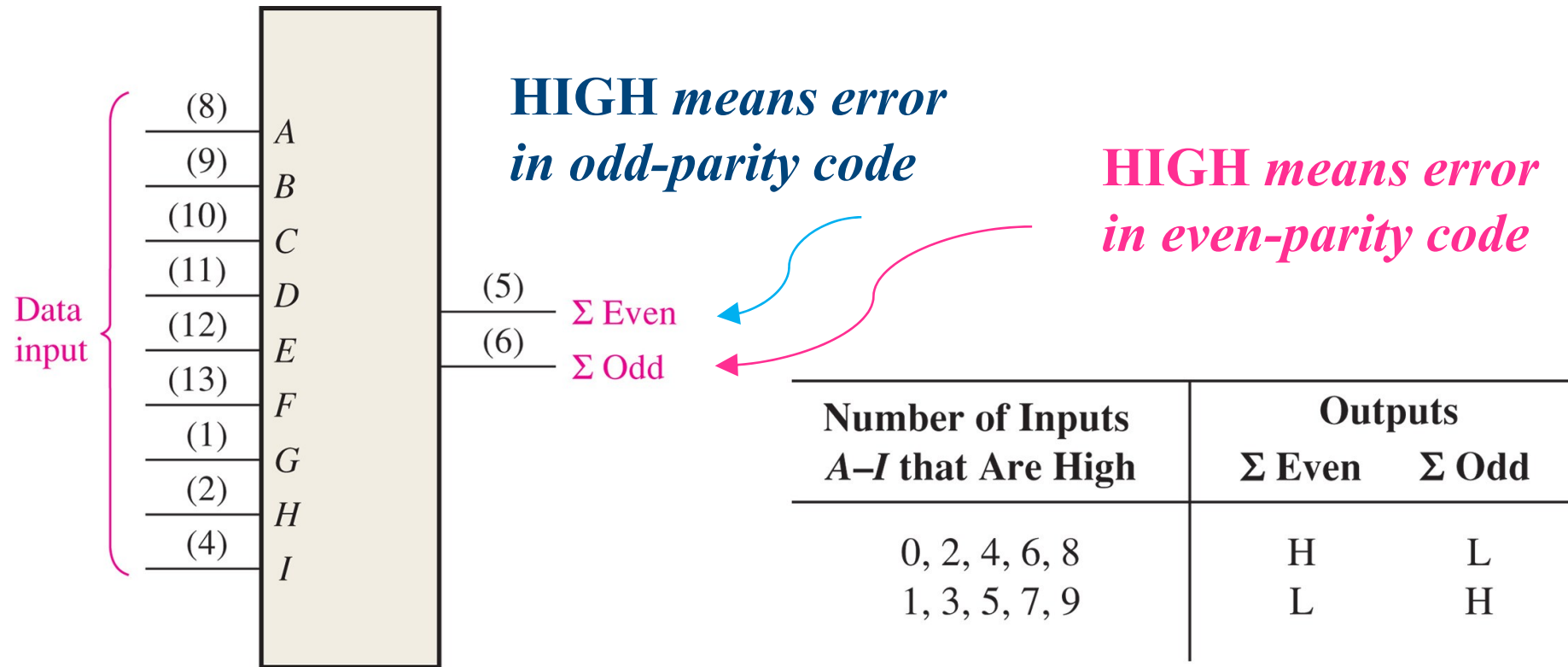
(a) Summing of two bits



(b) Summing of four bits

(2) IC Applications: 74HC280 Parity Generator/Checker and Serial Transmission

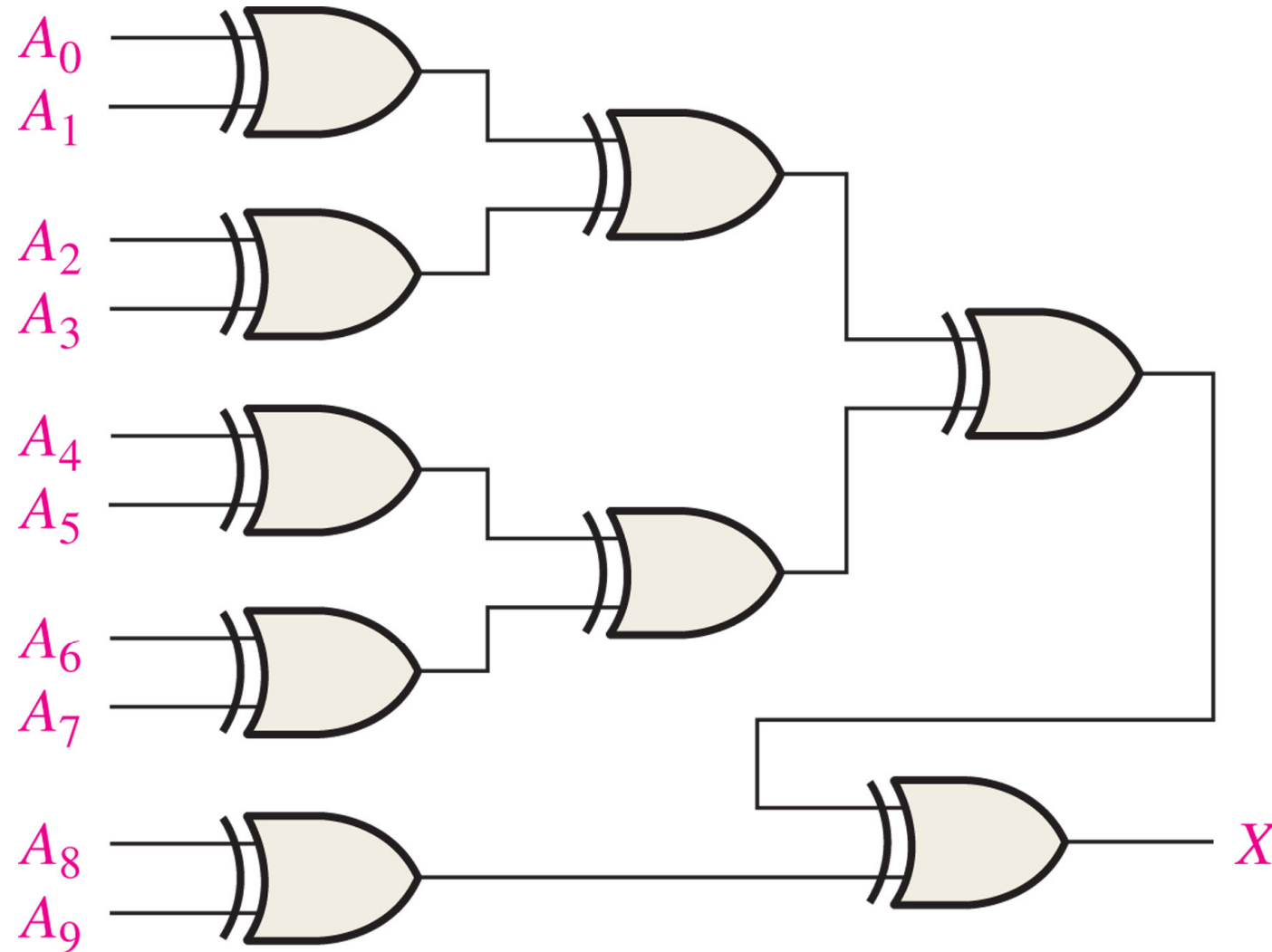
- Check for odd/even parity on a 9-bit code (8 data bits and 1 parity bit)
- Generate a parity bit for a code with up to 9 bits
- Use the output Even (Odd) as parity bit to generate odd (even) parity



(a) Traditional logic symbol

(b) Function table

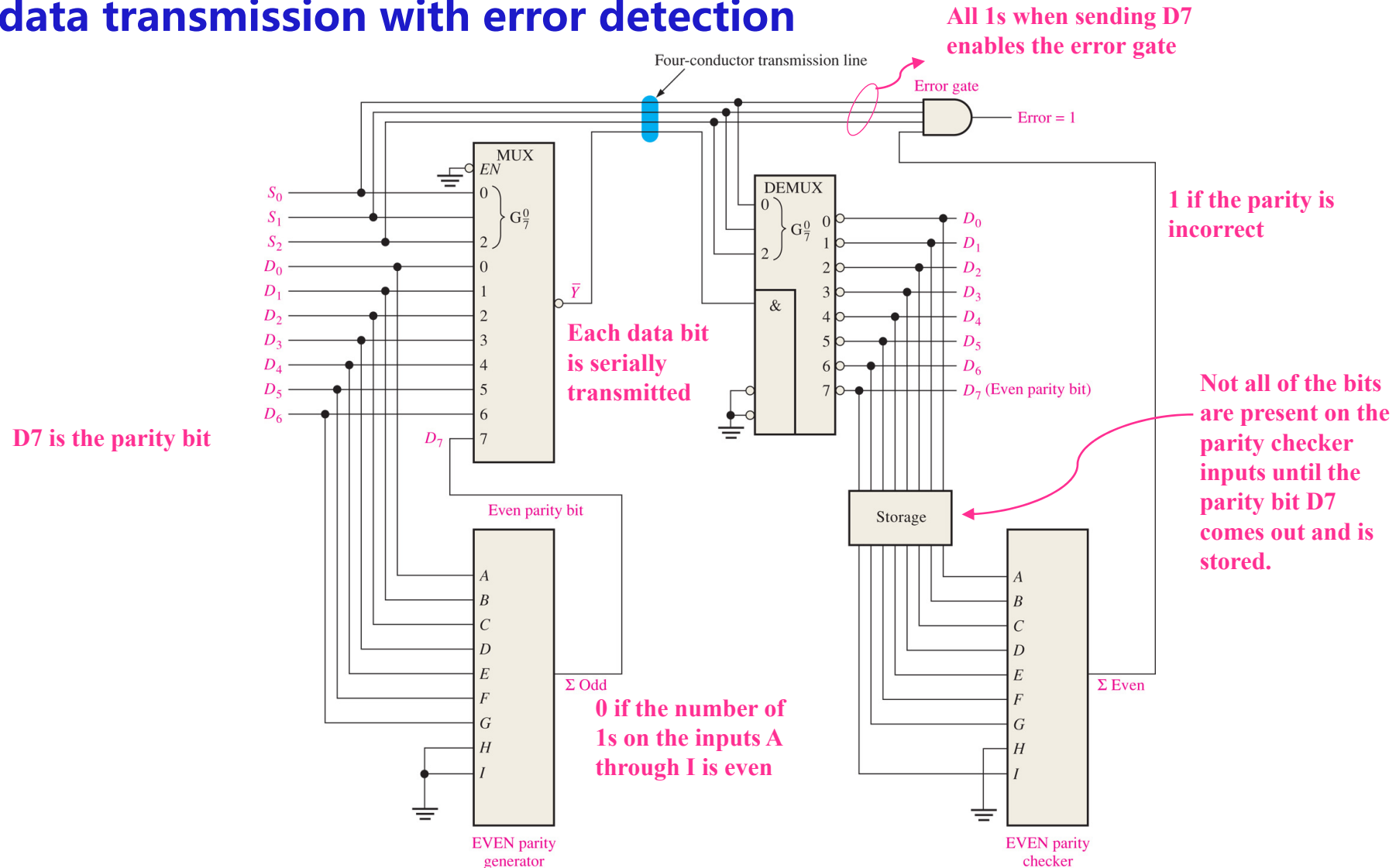
(2) IC Applications: 74HC280 Parity Generator/Checker and Serial Transmission



(2) IC Applications: 74HC280 Parity Generator/Checker and Serial Transmission



Serial data transmission with error detection



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7

S_0

S_1

S_2

Data stream at DEMUX input

D_0 D_1 D_2 D_3 D_4 D_5 D_6 P D_0 D_1 D_2 D_3 D_4 D_5 D_6 P

Error

Bit received incorrectly (0 was transmitted)

$PD_6D_5D_4D_3D_2D_1D_0$

$0\ 1\ 0\ 1\ 0\ 1\ 0\ 1$

Parity bit

$PD_6D_5D_4D_3D_2D_1D_0$

$0\ 1\ 1\ 1\ 0\ 1\ 0\ 1$

Parity bit

Error in D_5

Even parity violated

Same code transmitted twice, one without error and one with error

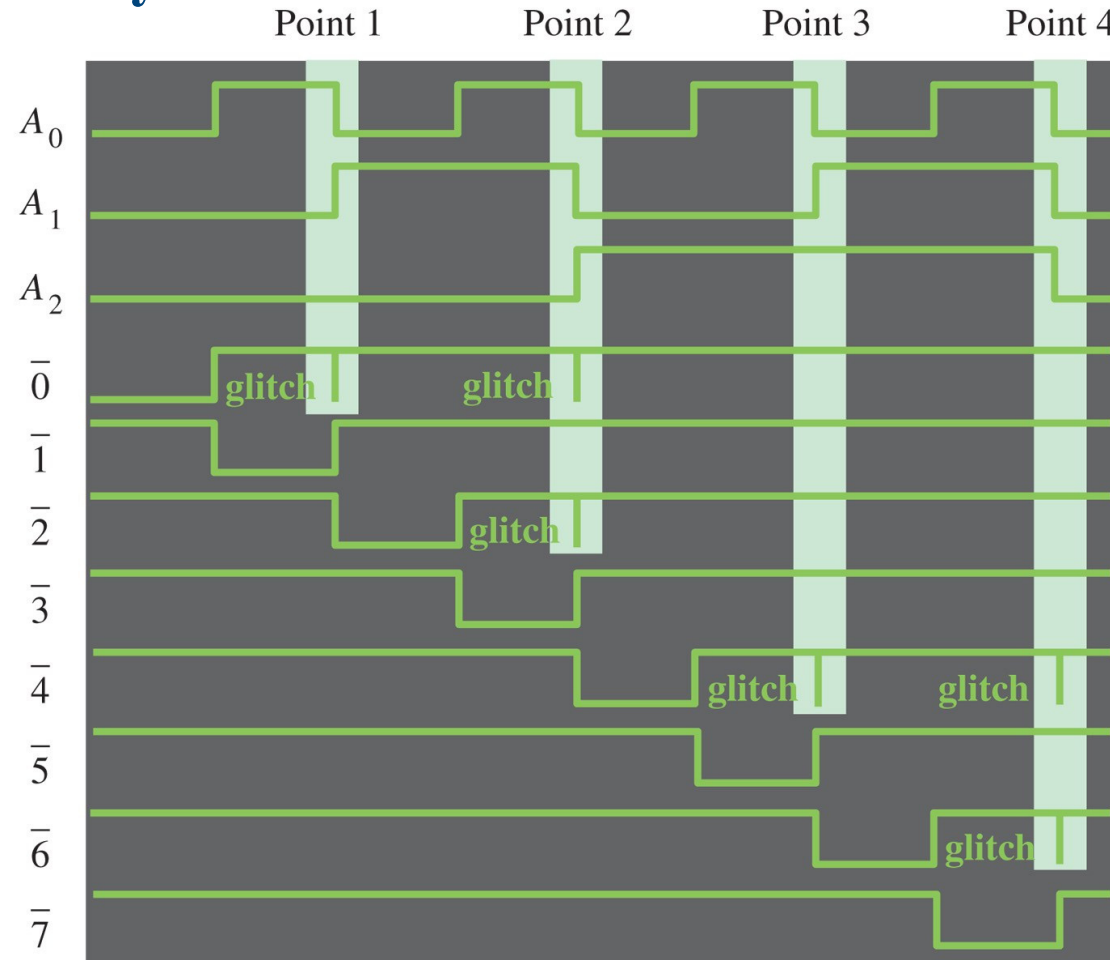
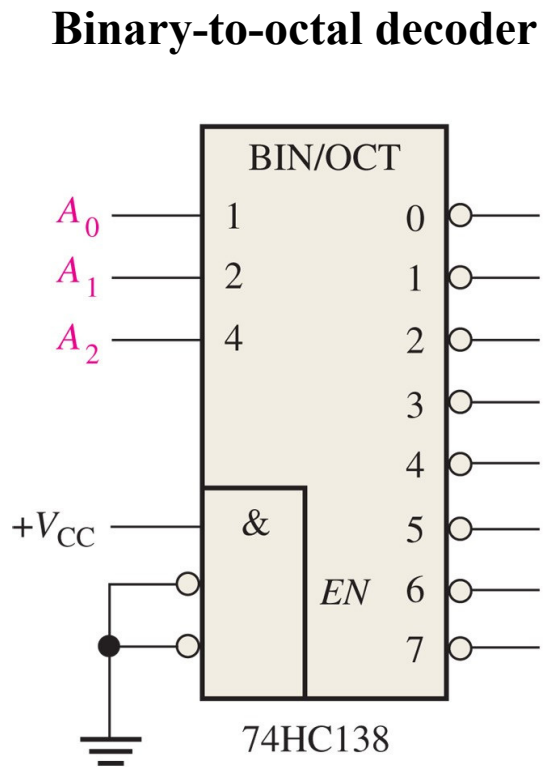
Main Content

Managing Signal Integrity in Digital Circuits

- Decoder Waveforms with Glitches
- Causes of Glitches During Input Transitions
- Strobe Pulse Application for Glitch Elimination

(1) Decoder Waveforms with Glitches

- A glitch: unwanted voltage spike (pulse) with very short duration
- Typically appear during signal transitions
- Caused by propagation delays

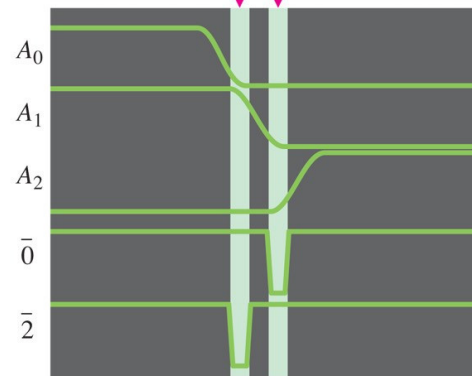


(2) Causes of Glitches During Input Transitions

Close up view of decoder input waveforms showing how transitional input states produce glitches in the output waveforms

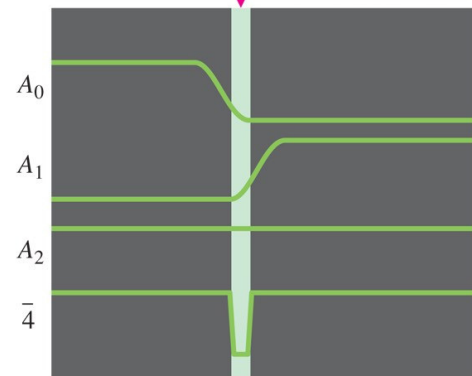
$A_2A_1A_0$ momentarily becomes first 010 and then 000 during transition from 011 to 100 due to delays in both A_1 and A_2

Point 2: waveforms on expanded time base
 A_1 HIGH; A_0, A_2 LOW → A_0, A_1, A_2 LOW



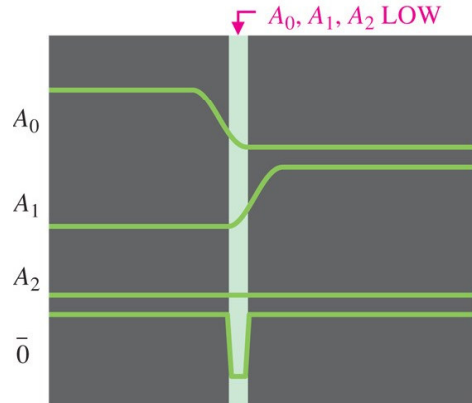
$A_2A_1A_0$ momentarily becomes 100 during transition from 101 to 110 due to delays in A_1

Point 3: waveforms on expanded time base
 A_0, A_1 LOW; A_2 HIGH



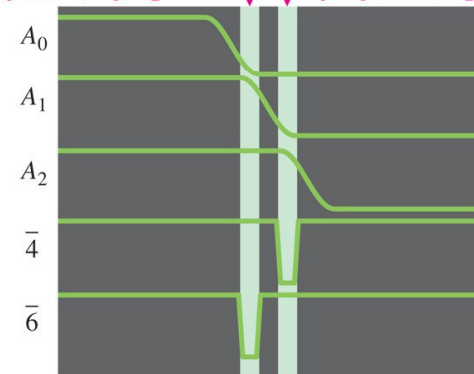
$A_2A_1A_0$ momentarily becomes 000 during transition from 001 to 010 due to delay in A_1

Point 1: waveforms on expanded time base
 A_0, A_1, A_2 LOW



$A_2A_1A_0$ momentarily becomes first 110 and then 100 during transition from 111 to 000 due to delays in both A_1 and A_2

Point 4: waveforms on expanded time base
 A_0 LOW; A_1, A_2 HIGH → A_0, A_1 LOW; A_2 HIGH

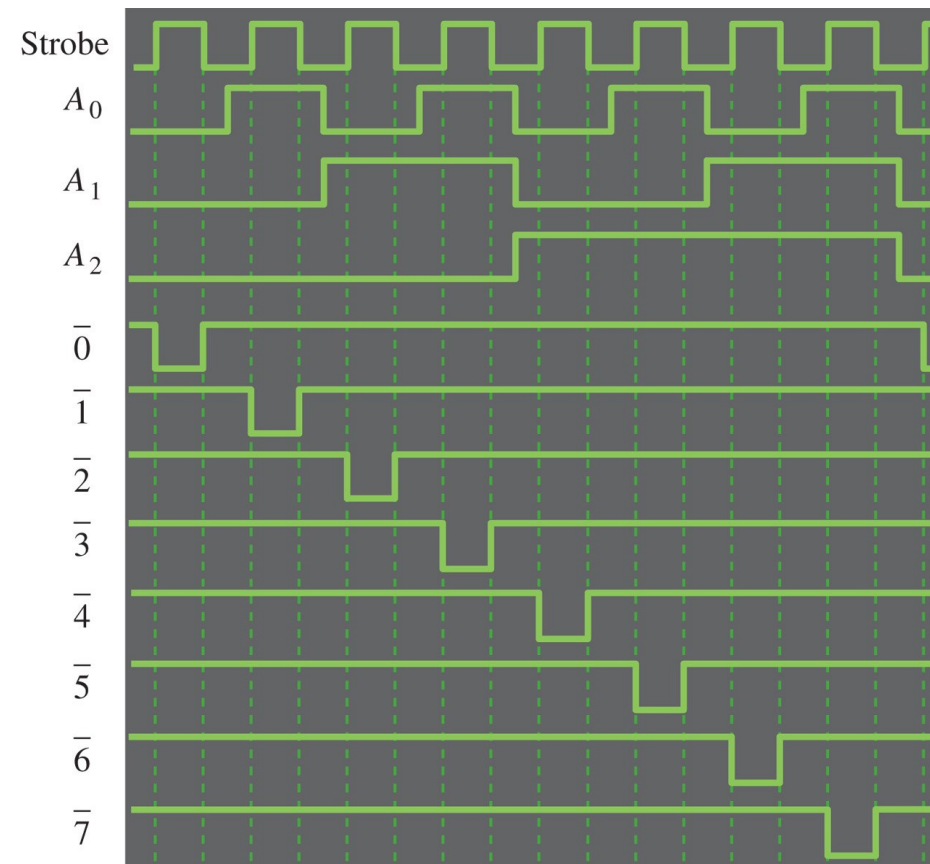
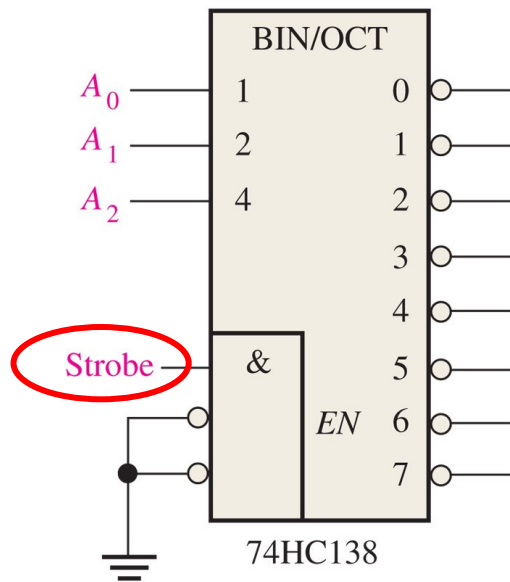


000

110 100

(3) Strobe Pulse Application for Glitch Elimination

- Application of a strobe pulse to eliminate glitches on decoder outputs
- Strobe pulse enables the decoder only during the times when the waveforms are NOT in transition
- Strobe corresponds to CLOCK signal in SEQUENTIAL logic



Main Content

Combinational Logic Application: Traffic Light System

- System Requirements and State Diagram (Gray Code Transitions)
- Combinational Logic for Light Outputs (Red, Yellow, Green)
- Trigger Logic for Timers (25s and 4s Intervals)

(1) System Requirements and State Diagram

Traffic Signal Controller

Develop a control logic for a traffic signal at the intersection of a busy main street and a lightly used side street.



- The main street green light will stay on for a minimum of 25 s OR **as long as there is no vehicle on the side street.** $(T_L + V_S)^-$
- The side street green light will stay on until there is no vehicle on the side street up to a maximum of 25 s. $(T_L V_S)$
- The yellow caution light will stay on for 4 s between changes from green to red on both the main street and the side street. T_S

V_S *A vehicle exists on the side street*

T_L *The 25s timer (long timer) is ON*

T_S *The 4s timer (short timer) is ON*

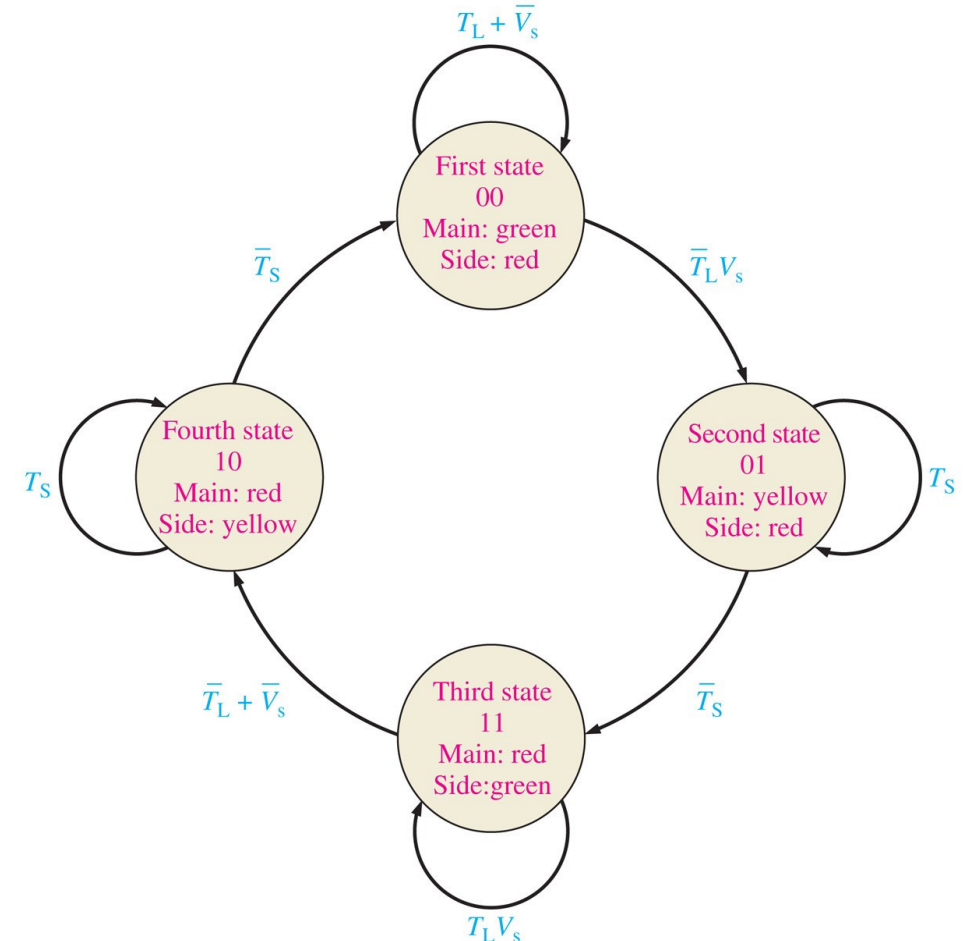
A complemented variable indicates the opposite condition.

(1) System Requirements and State Diagram



State diagram

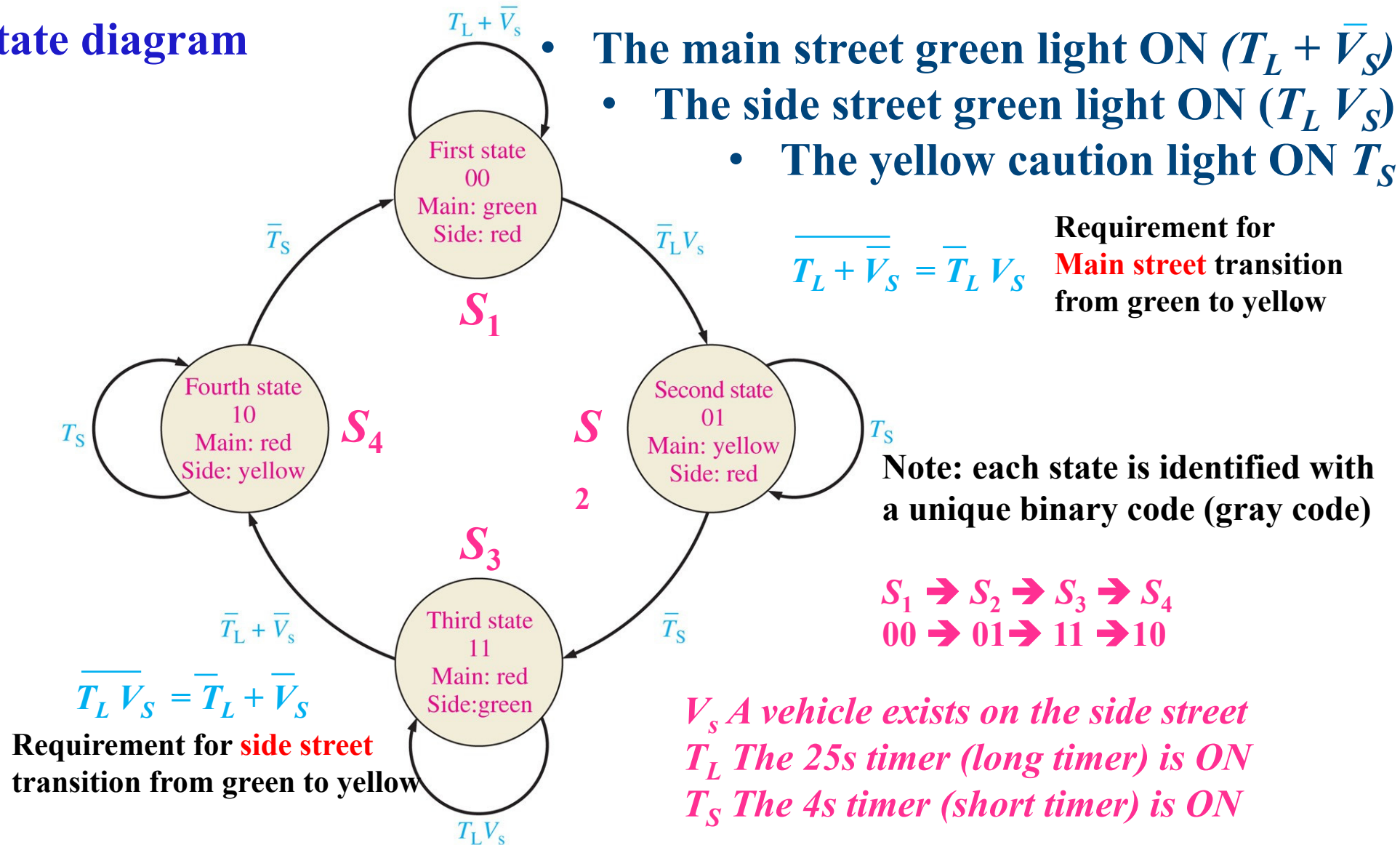
- Describes the complete operation.
- Shows the sequence of states, the conditions for each state, and the requirements for transitions from one state to the next.
- A looping arrow means that the system remains in a state
- An arrow between states means the system transitions to the next state.
- Boolean expression or variable on each arrow indicates the condition under which the system remains in a state or transitions to the next.



(1) System Requirements and State Diagram

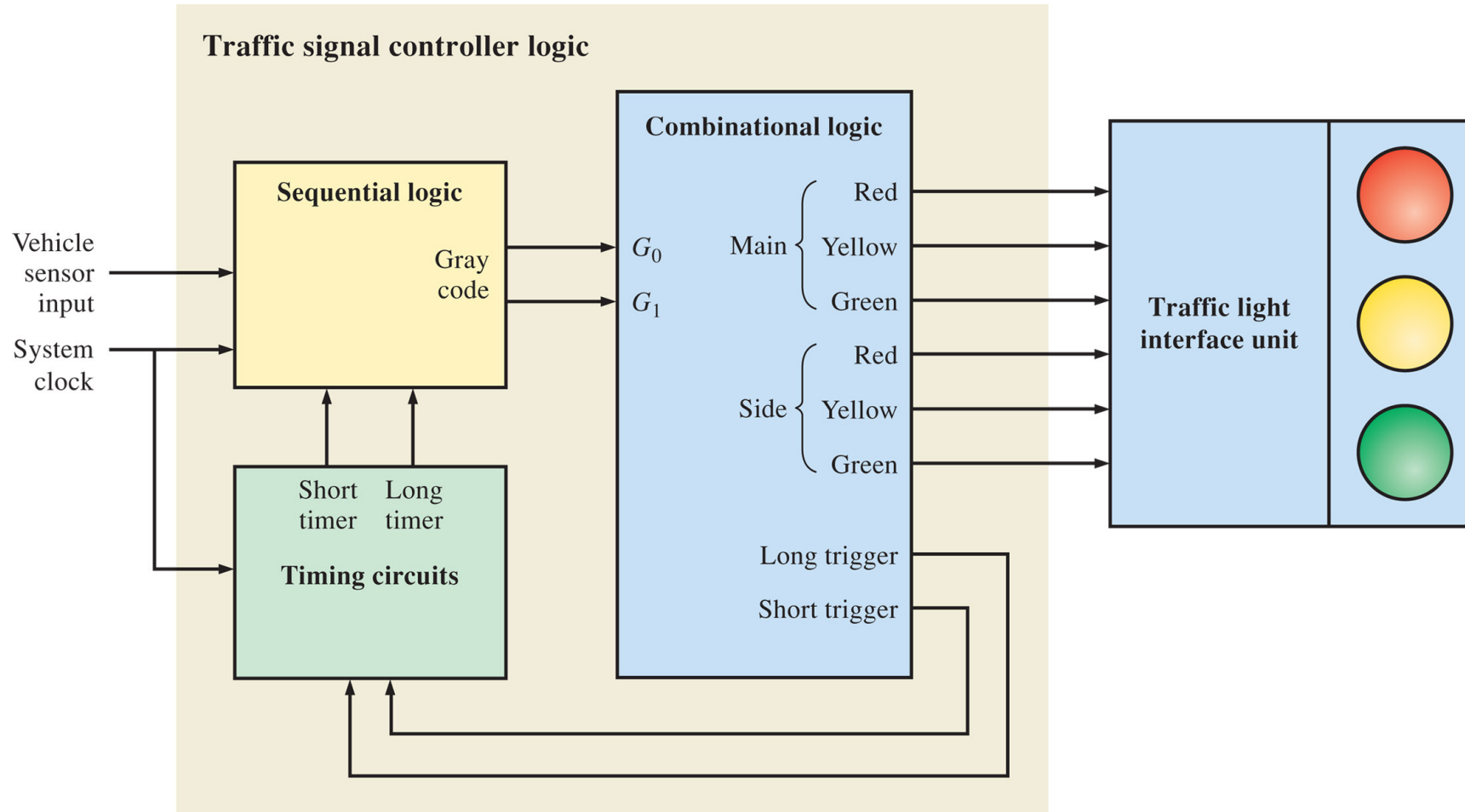


State diagram



(2) Combinational Logic for Light Outputs

Block diagram



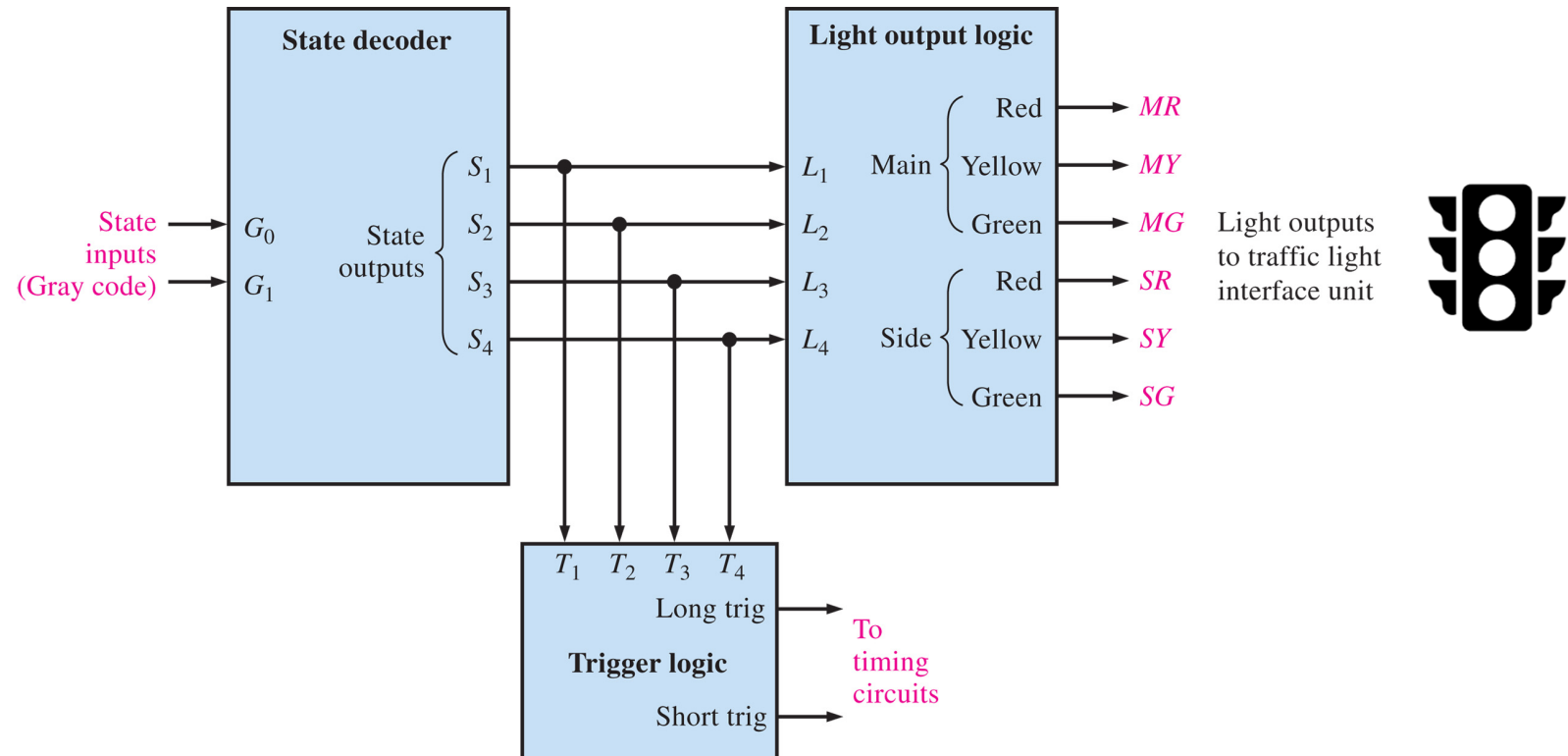
(2) Combinational Logic for Light Outputs



Combinational Logic diagram

- Provides outputs to turn the signal lights ON and OFF.
- Also provides trigger outputs to START the long and short timers.

The input sequence to this logic represents the four states described by the state diagram.



(2) Combinational Logic for Light Outputs



State Decoder Truth Table

$00 \rightarrow 01 \rightarrow 11 \rightarrow 10$
 $S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4$

$$\begin{aligned} S_1 &= \overline{G_1} \overline{G_0} \\ S_2 &= \overline{G_1} G_0 \\ S_3 &= G_1 G_0 \\ S_4 &= G_1 \overline{G_0} \end{aligned}$$

Each output can be expressed by only one minterm

Truth table for the state decoder.

State Inputs (Gray Code)		State Outputs			
G_1	G_0	S_1	S_2	S_3	S_4
0	0	1	0	0	0
0	1	0	1	0	0
1	1	0	0	1	0
1	0	0	0	0	1

(2) Combinational Logic for Light Outputs

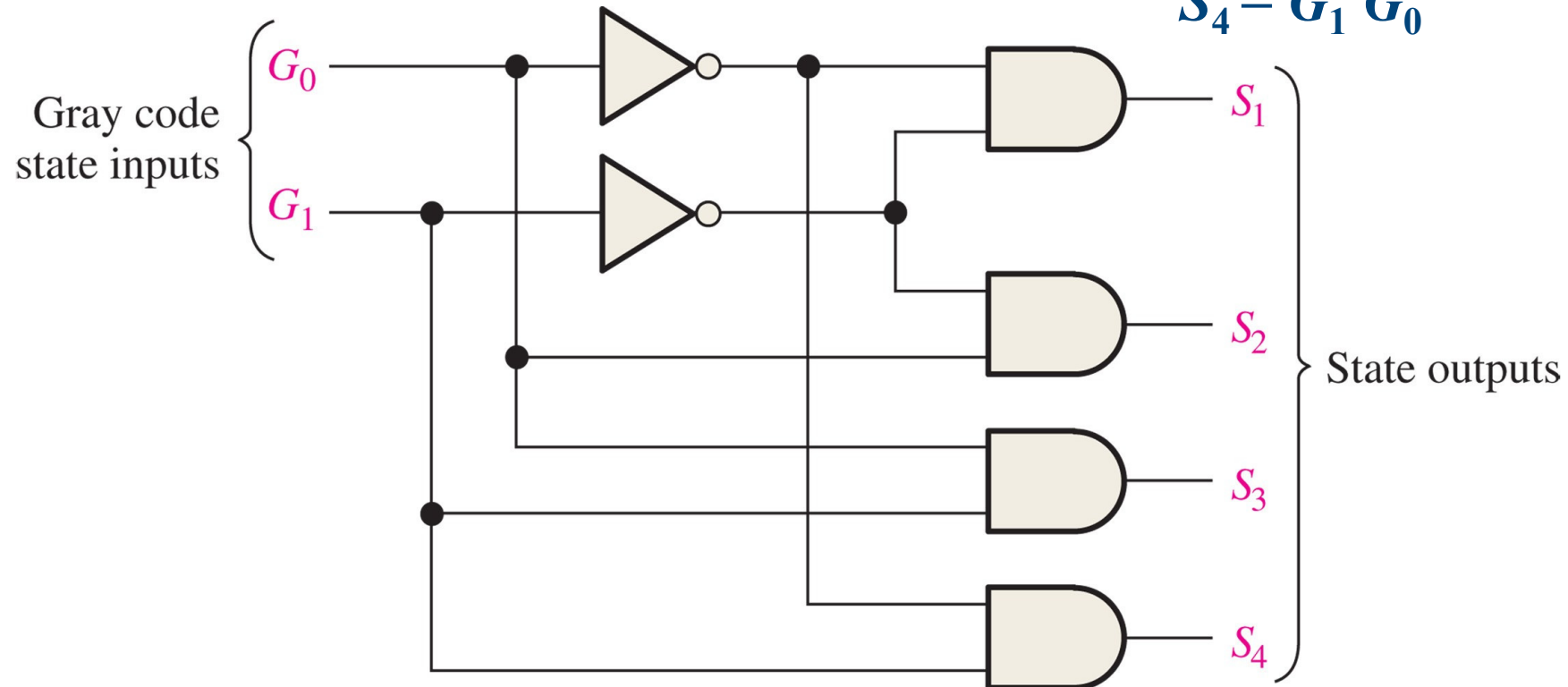


THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

State Decoder Logic

This logic decodes the 2-bit Gray code from the sequential logic to determine which of the four states the system is in.

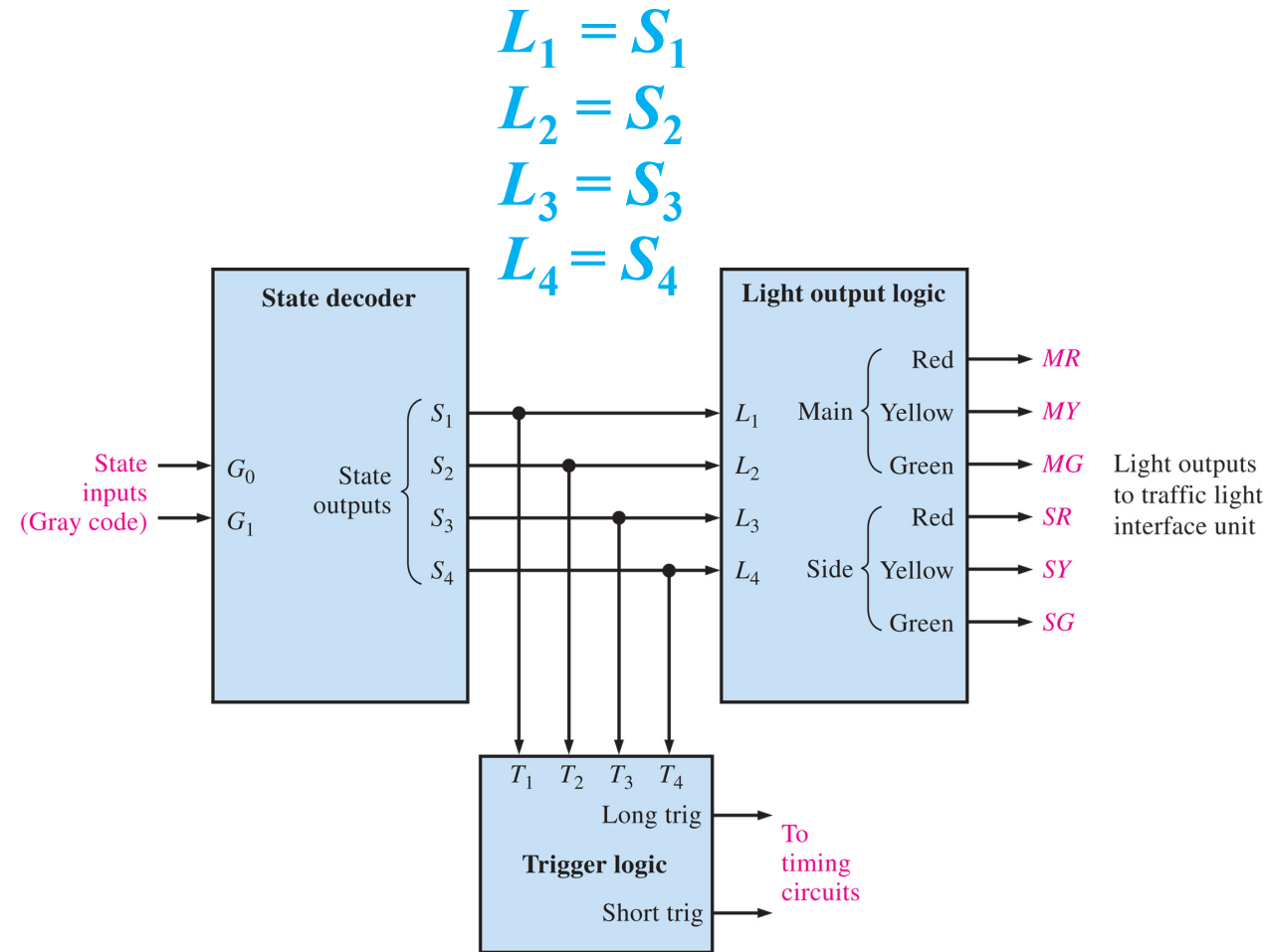
$$\begin{aligned} S_1 &= \overline{G_1} \overline{G_0} \\ S_2 &= \overline{G_1} G_0 \\ S_3 &= G_1 \overline{G_0} \\ S_4 &= G_1 G_0 \end{aligned}$$



(2) Combinational Logic for Light Outputs



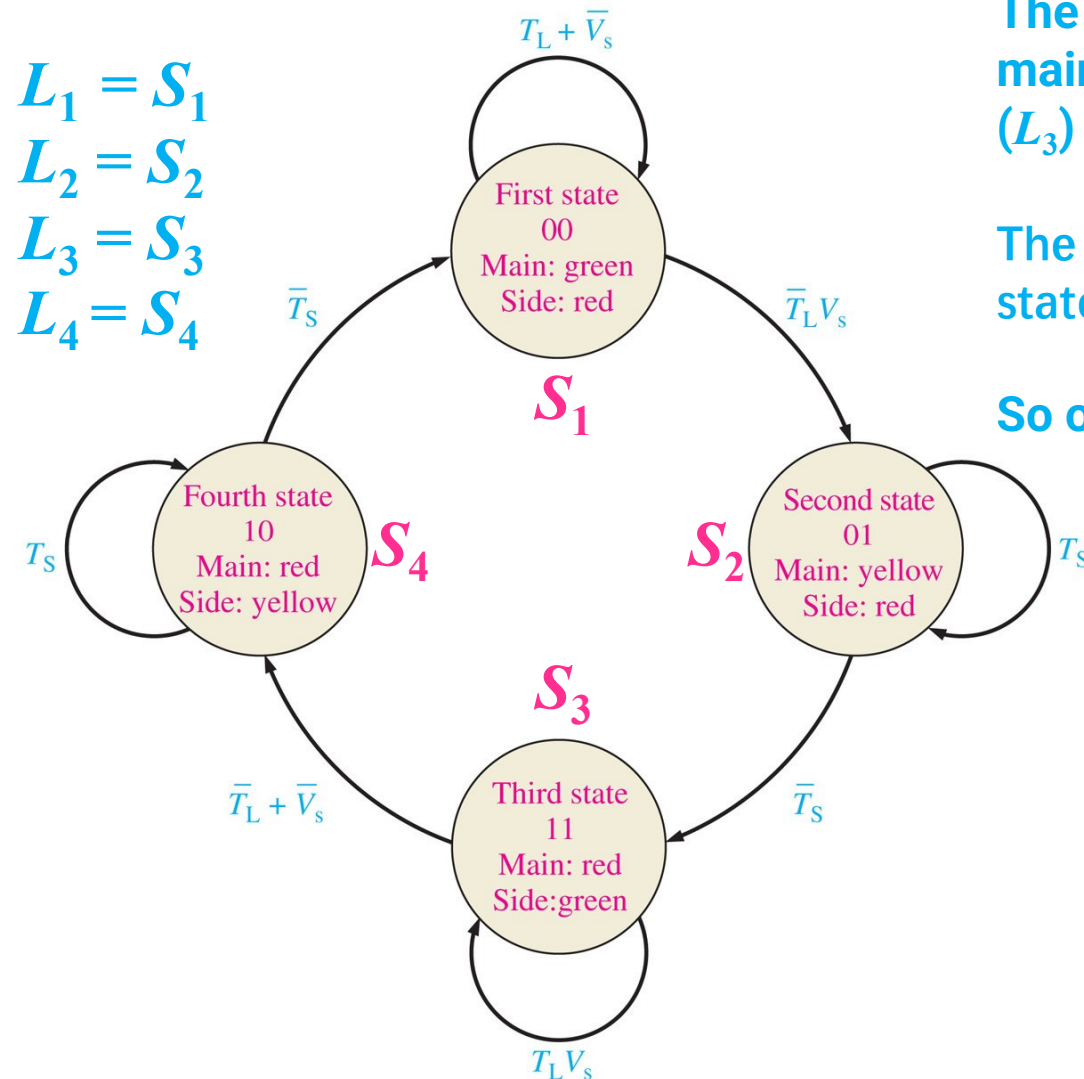
Light Output Logic



(2) Combinational Logic for Light Outputs



Light Output Logic



The state diagram shows that the main red (MR) is on in the third state (L_3) or in the fourth state (L_4)

The main yellow is on in the second state (L_2)

So on so forth

$$MR = L_3 + L_4$$

$$MY = L_2$$

$$MG = L_1$$

$$SR = L_1 + L_2$$

$$SY = L_4$$

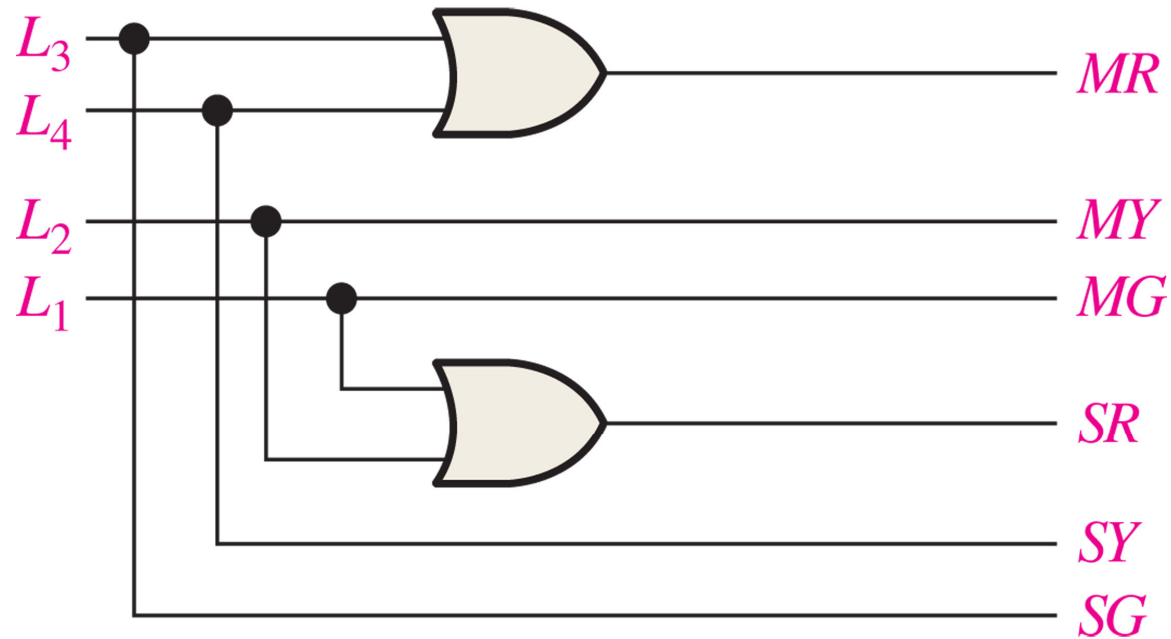
$$SG = L_3$$

(2) Combinational Logic for Light Outputs



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

Light Output Logic



$$MR = L_3 + L_4$$

$$MY = L_2$$

$$MG = L_1$$

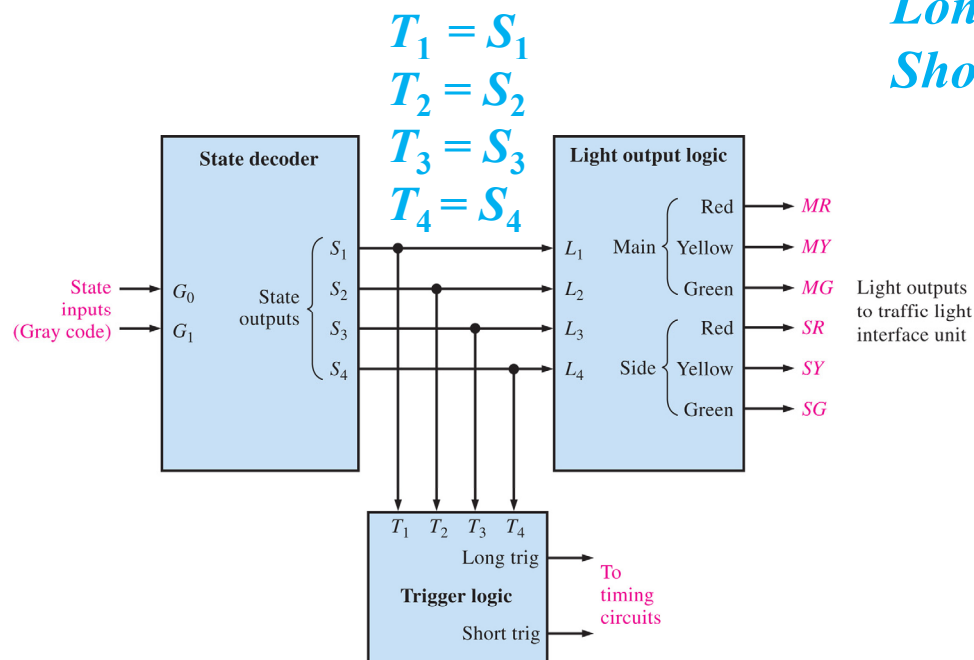
$$SR = L_1 + L_2$$

$$SY = L_4$$

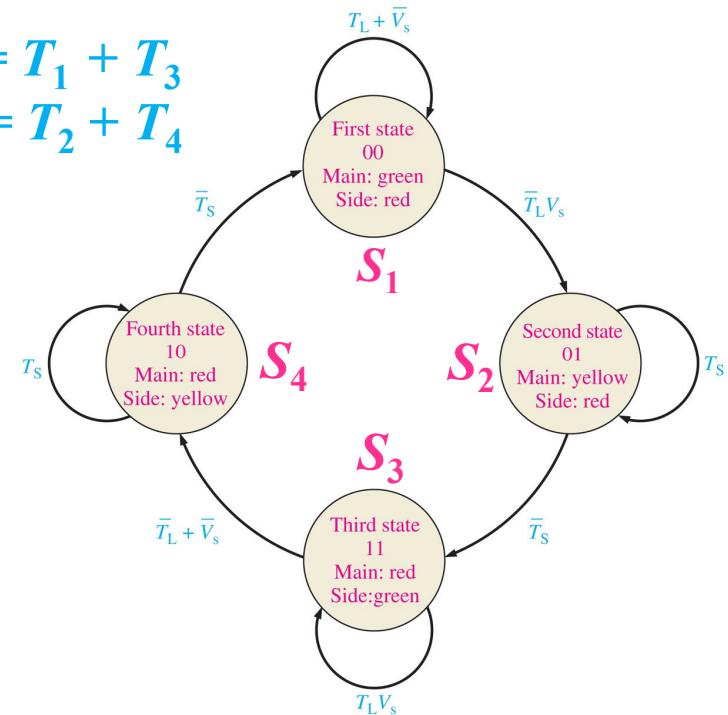
$$SG = L_3$$

(3) Trigger Logic for Timers

- Produces the long trigger output and the short trigger output.
- The long trigger output initiates the 25 s timer on a LOW-to-HIGH transition at the beginning of the first or third states.
- The short trigger output initiates the 4 s timer on a LOW-to-HIGH transition at the beginning of the second or fourth states.
- The Boolean expressions for this logic are



$$\text{Long trig} = T_1 + T_3$$
$$\text{Short trig} = T_2 + T_4$$

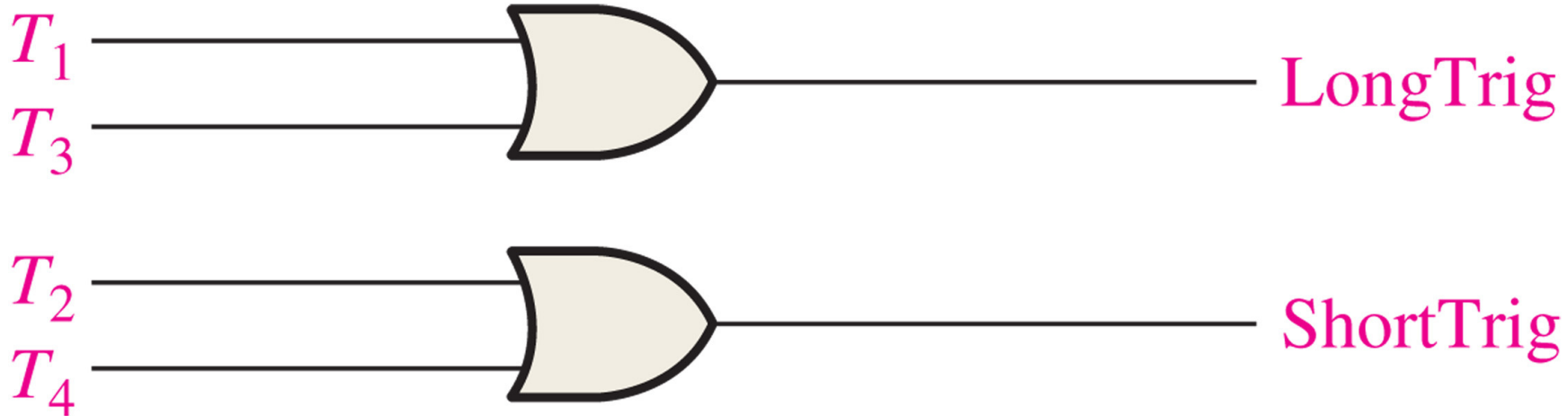


(3) Trigger Logic for Timers

Trigger Logic

$$\text{Long trig} = T_1 + T_3$$

$$\text{Short trig} = T_2 + T_4$$





香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Thank you for listening !

